

GRAFIKA KOMPUTEROWA I KOMUNIKACJA Z KOMPUTEREM

Wykład 6

Andrzej Śluzek

pokój: 3/64

email: andrzej_sluzek@sggw.edu.pl



Tematyka wykładów i zajęć laboratoryjnych

(oficjalny sylabus, kolejność i tematyka może być nieco inna)

1. Wstęp. Urządzenia graficzne wejścia-wyjścia.
2. Podstawowe pojęcia i definicje dotyczące obrazów cyfrowych (w technice rastrowej i wektorowej) oraz modeli kolorów.
3. Punktowe, lokalne i globalne metody przekształcania obrazów.
4. Transformacje geometryczne obiektów graficznych i wypełnianie figur.
5. Eliminacja elementów zasłoniętych, metody kreślenia krzywych.
6. Modele oświetlenia i cieniowania.
7. Przekształcenia morfologiczne i ich zastosowania.
8. Transformaty obrazów i sygnałów dźwiękowych (FFT, **Hough, Radon**). Wybrane metody filtracji danych jedno- i dwuwymiarowych.
9. **Wybrane modele interakcji człowiek-komputer. Zasady projektowania i analizy interfejsów komputerowych.**
10. Interfejsy dla osób niepełnosprawnych. Interakcja człowiek-komputer dla komputerów przyszłości.

Transformacje Hough'a i Radona

Wstęp

Matematycznie, *transformata Hough'a* i *transformata Radona* są ze sobą „spokrewnione” (*transformata Radona* jest uogólnieniem jednej z *transformat Hough'a*). Są one jednak używane są do zupełnie innych celów.

Transformata Hough'a służy do znajdowania w obrazach linii o określonym kształcie (np. linii prostych lub okręgów) a tym samym używana jest do tworzenia „poprawionych” wersji obrazów, w których te linie oryginalnie mogły być nie w pełni widoczne lub zaszumione.

Natomiast *transformata Radona* jest przede wszystkim jednym z fundamentalnych narzędzi *tomografii komputerowej*, gdzie służy do rekonstrukcji przekrojów obiektów (np. ciała ludzkiego).



Transformata Hough'a

Transformata Hough'a w praktyce stosowana jest wyłącznie do obrazów konturowych i jej zadaniem jest aproksymowanie w możliwie najlepszy sposób zbiorów punktów konturowych liniami z danej rodziny (tzn. wykrycie takich linii i znalezienie optymalnych parametrów ich równań).

Mówiąc ogólnie, transformata Hough'a przekształca jeden (konturowy) pixel obrazu w $(N-1)$ -wymiarową powierzchnię w N -wymiarowej przestrzeni parametrycznej (gdzie N jest liczbą parametrów opisujących daną rodzinę linii).

Na przykład, linie proste mają równanie postaci:

$$y = mx + b$$

tzn. $N = 2$ (parametry m i b).

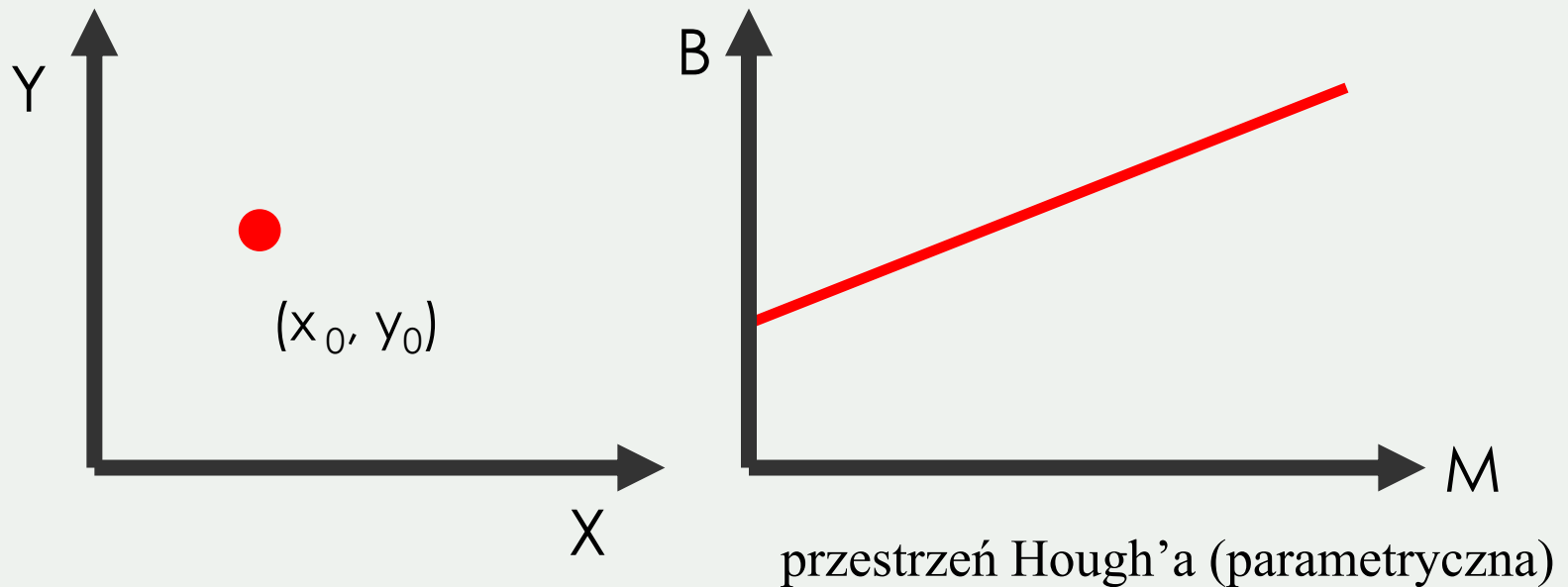
Transformata Hough'a

Jeśli dany jest pixel o współrzędnych (x_0, y_0) , każda linia prosta przechodząca przez ten pixel musi spełniać równanie:

$$y_0 = mx_0 + b$$

W przestrzeni parametrów $M \times B$ uzyskujemy więc równanie prostej:

$$b = -x_0m + y_0$$

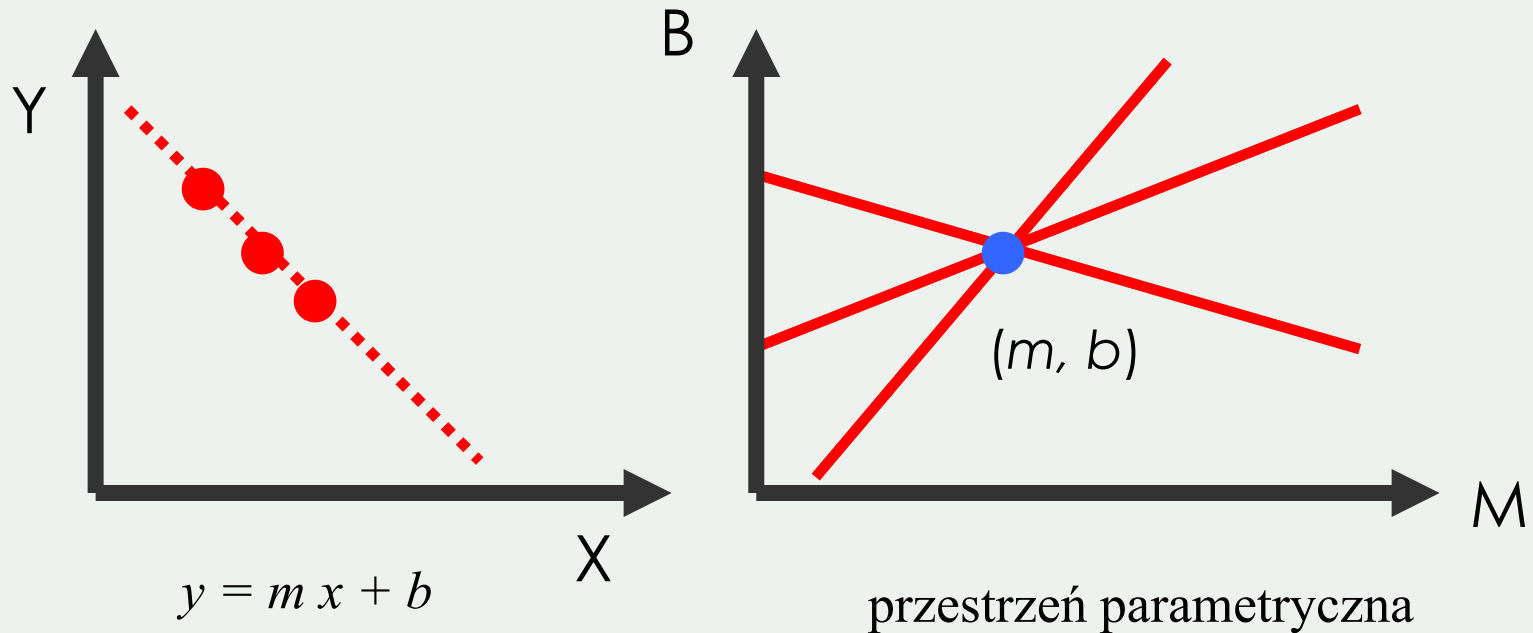


Transformata Hough'a

Natomiast zbiór pixeli ułożonych wzdłuż linii o równaniu

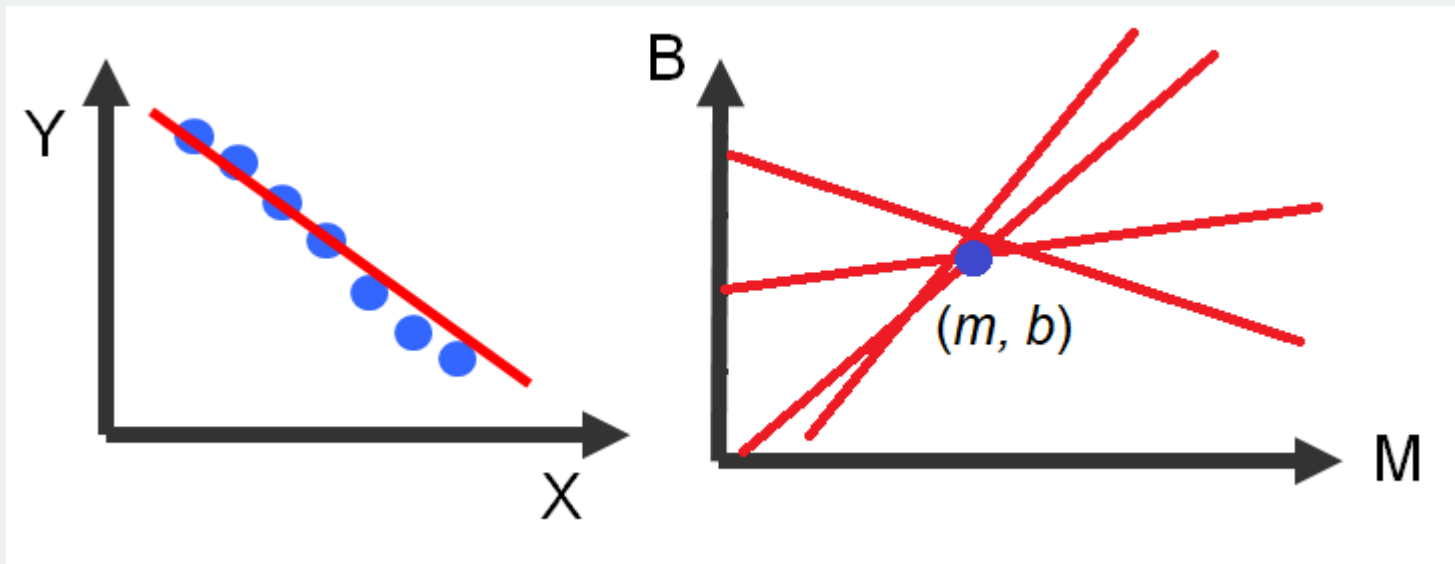
$$y = mx + b$$

generuje w przestrzeni parametrów pęk linii prostych, które przecinają się w punkcie o współrzędnych (m, b) .



Transformata Hough'a

Jeżeli pixele jedynie w przybliżeniu leżą wzdłuż linii prostej, linie odpowiadające tym pixelom przecinają się nie w jednym punkcie, a w zbiorze wielu punktów skupionych wokół (m, b) reprezentującego hipotetyczną linię, wzdłuż której ułożone są te pixele.



przestrzeń parametryczna

Transformata Hough'a

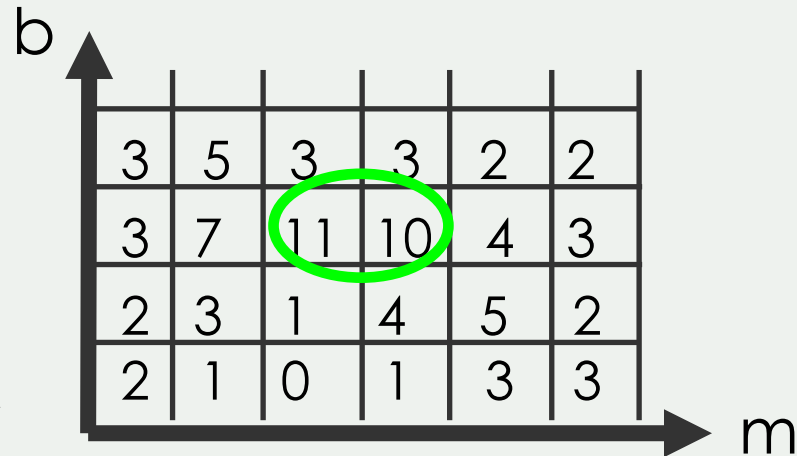
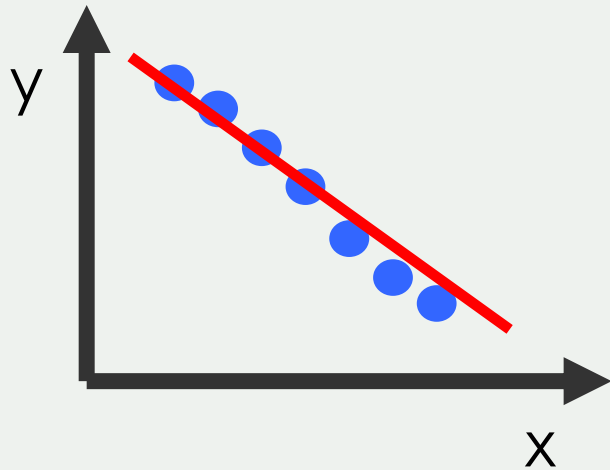
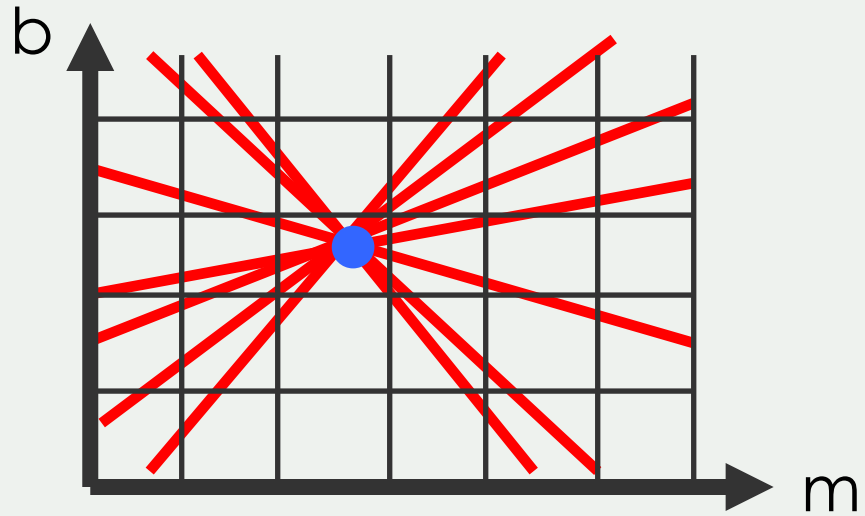
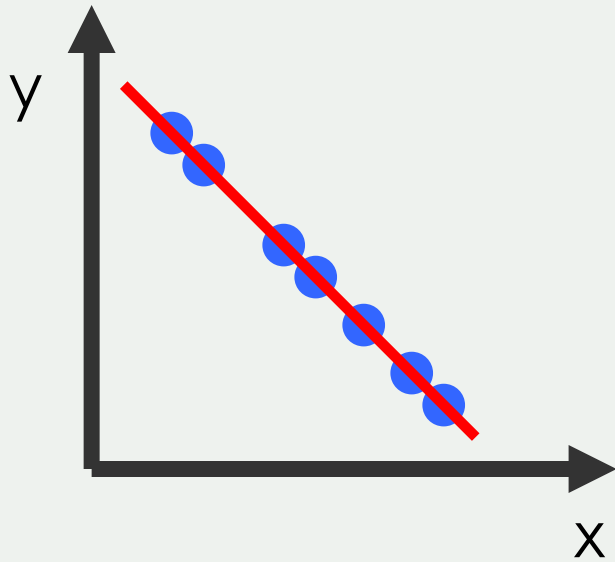
W praktyce, zarówno przestrzeń obrazu $X \times Y$, jak i przestrzeń parametrów $M \times B$ jest zdyskretyzowana

W przestrzeni $X \times Y$ są to pixele, a w przestrzeni $M \times B$ „komórki” reprezentujące wszystkie wartości parametrów (m, b) , które są zaokrąglone do wartości w środku danej „komórki”.

Liczbę par linii przecinających się wewnątrz danej komórki w przestrzeni $M \times B$ można w przybliżeniu aproksymować liczbą linii przecinających tę komórkę (następny slajd).

Potencjalne istnienie linii prostej w obrazie konturowym wykrywa się zatem znajdując „komórki” przecinane przez największą liczbę linii prostych w przestrzeni parametrów.

Transformata Hough'a



Liczby nie reprezentują
pokazanego zestawu pixeli.

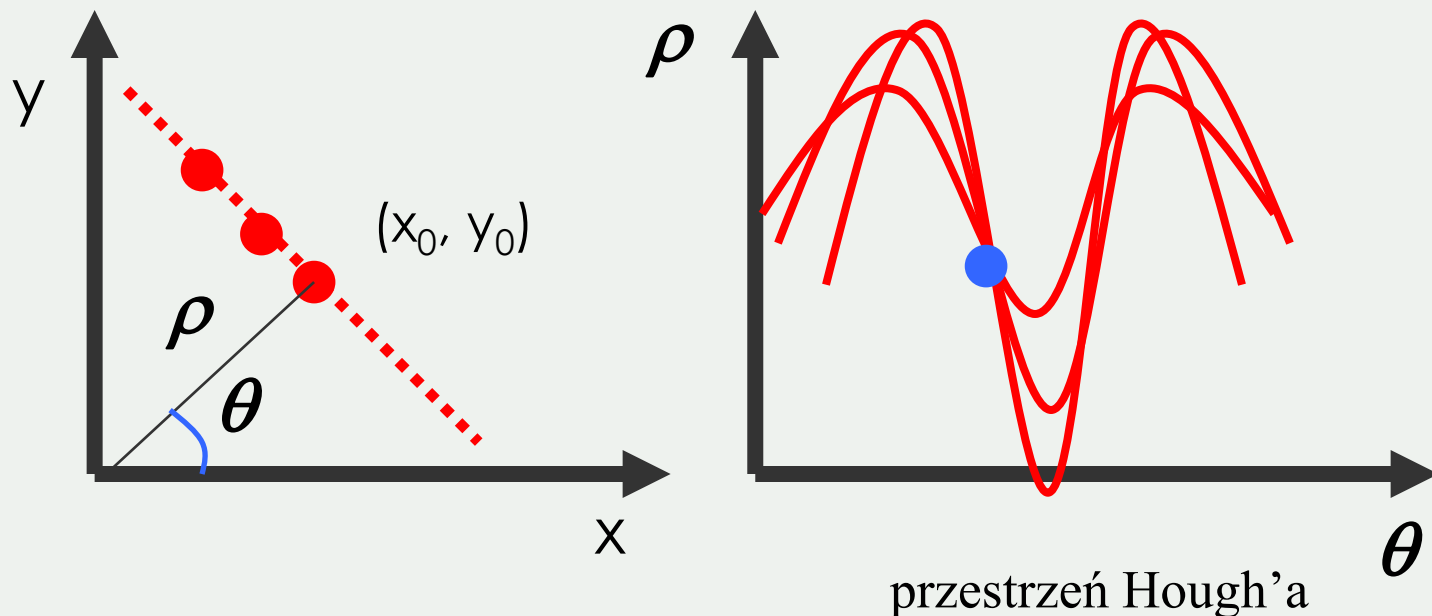
Transformata Hough'a

Problem: przestrzeń parametrów $m \times b$ jest nieograniczona (m może przyjmować wielkie wartości nawet w obrazach o ograniczonych rozmiarach).

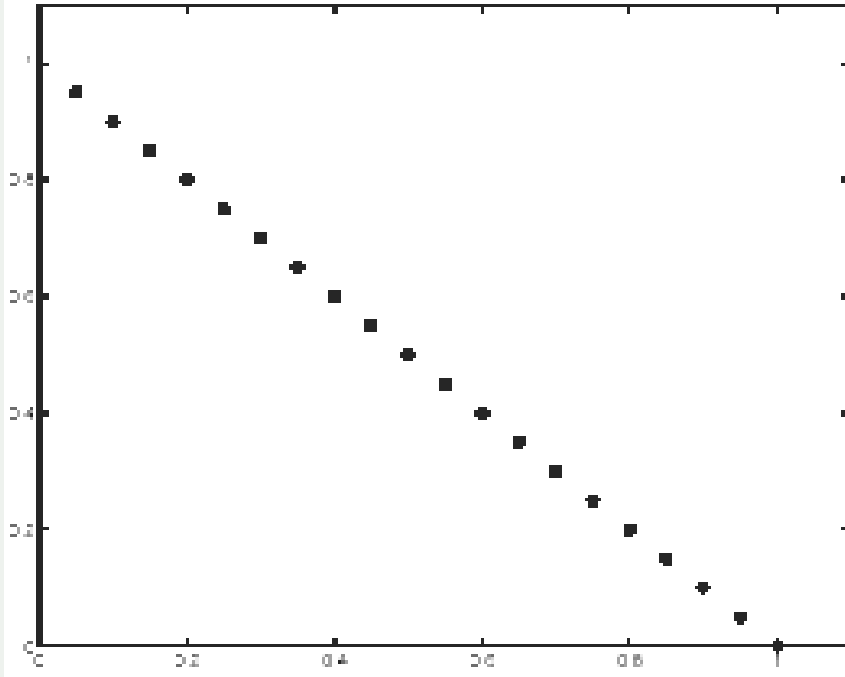
Korzystniej jest więc używać alternatywnego równania prostej:

$$x \cos \theta + y \sin \theta = \rho$$

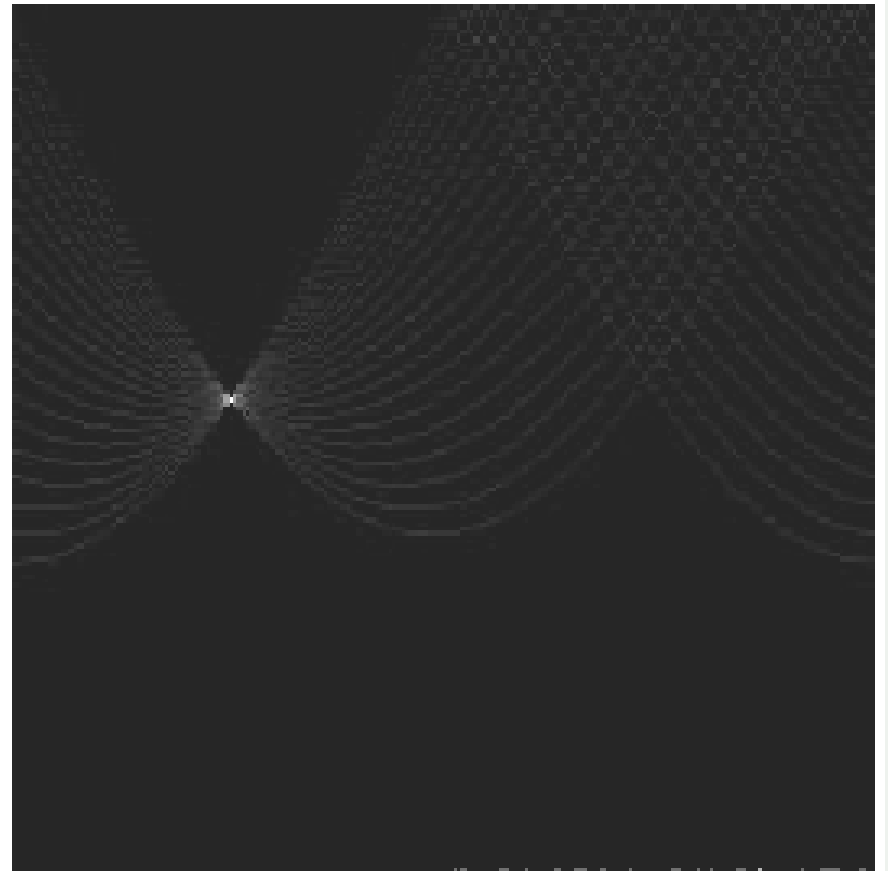
Wówczas każdy pixel (x_0, y_0) generuje w przestrzeni parametrów sinusoidę o równaniu $\rho = x_0 \cos \theta + y_0 \sin \theta$, a sinusoidy współliniowych pixeli przecinają w jednym punkcie przestrzeni parametrów.



Transformata Hough'a



pixele

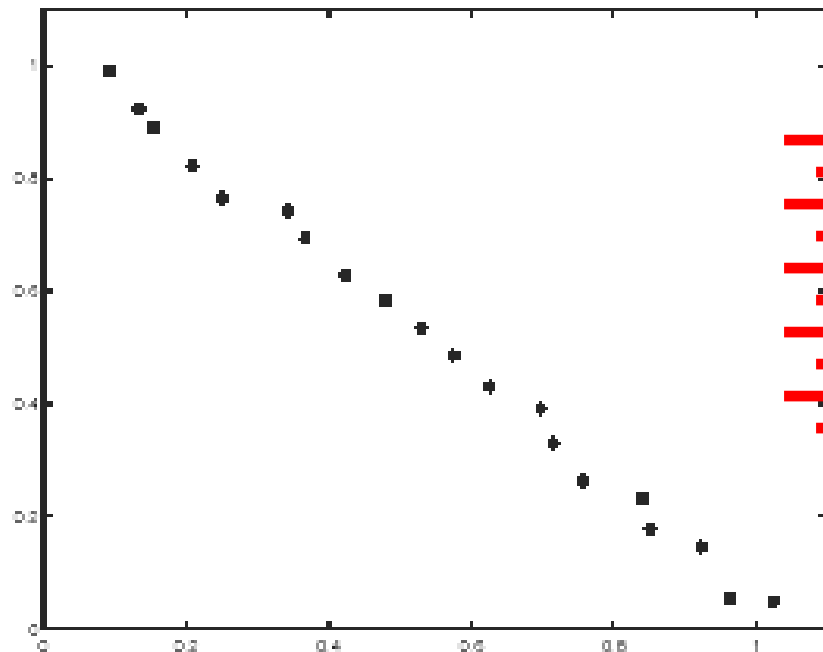


wyniki przecięć („głosowania”)

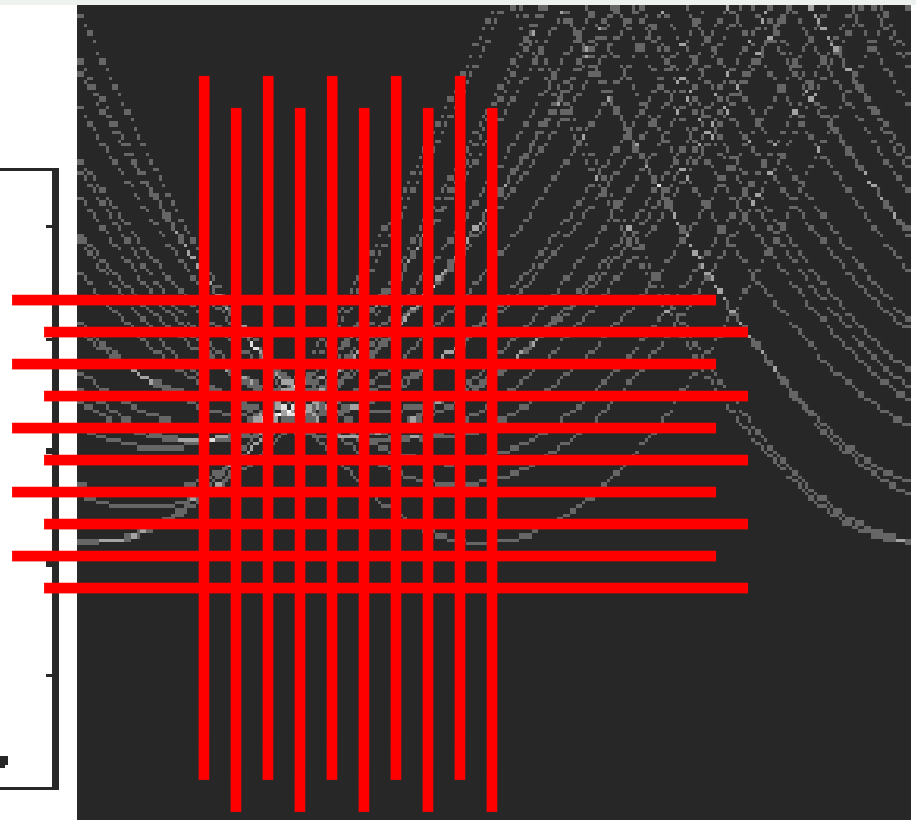
Ponieważ zakresy zarówno kąta θ jak i promienia ρ są skończone (przy skończonej rozdzielczości obrazu) więc można oba wymiary zdyskretyzować siatką prostokątną.

Transformata Hough'a

Niedokładna linia

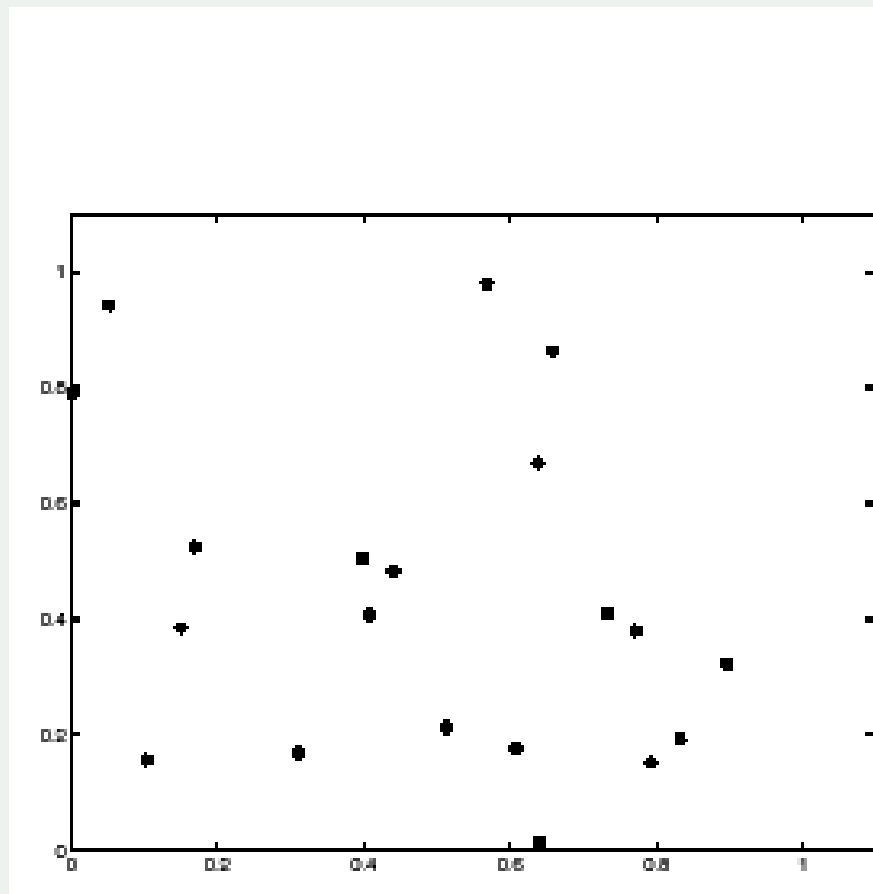


pixel

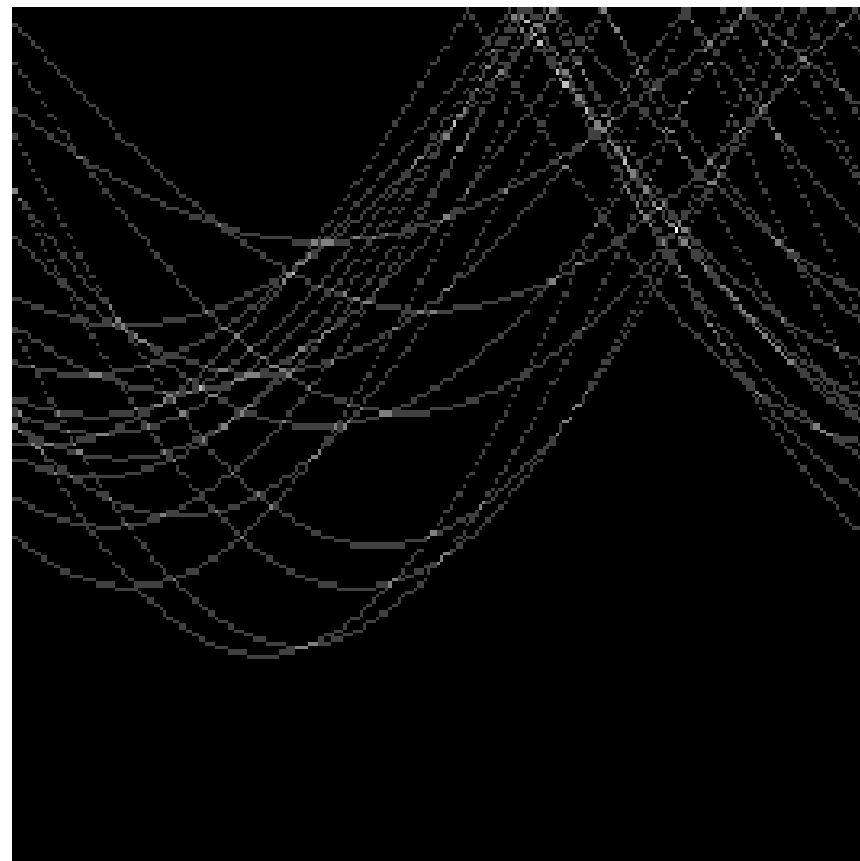


wyniki przecięć

Transformata Hough'a



pixele



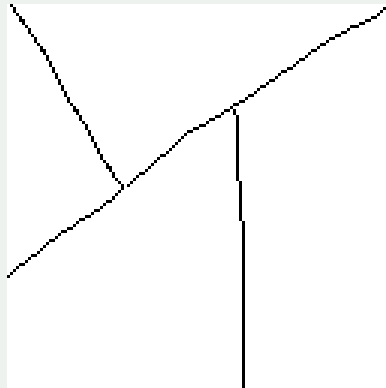
wyniki przecięć

W przypadku braku linii, mogą mimo wszystko pojawiać się przypadkowe (raczej niezbyt wybitne) lokalne maksima w przestrzeni Hough'a.

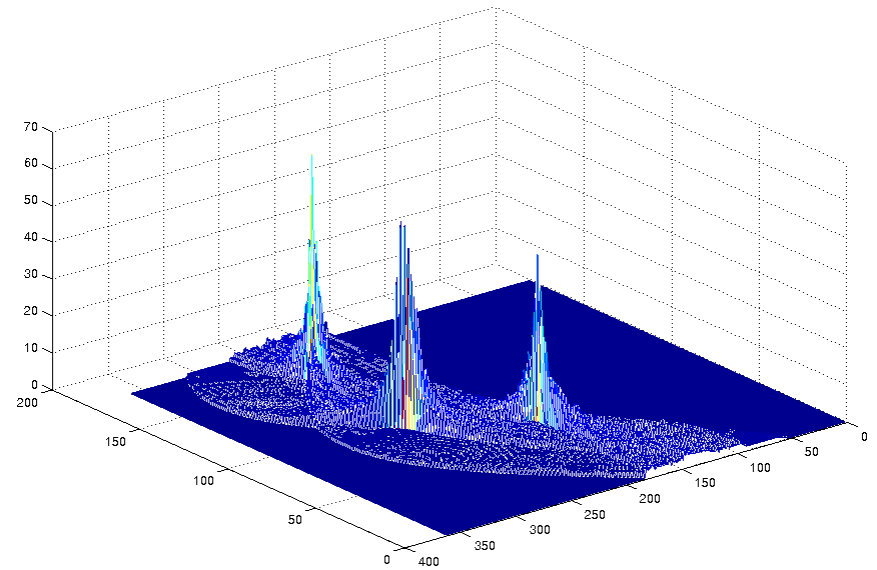
Transformata Hough'a



obraz



detekcja konturów



wyniki przecięć

Transformata Hough'a

Podsumowanie dla linii prostych

- Możliwość użycia parametrów m, b (problemy z potencjalnie nieskończoną siatką dyskretyzacji).
- Zalecane używanie parametrów ρ, θ (skończona siatka dyskretyzacji).
- Inne istotne zagadnienia
 - Rozmiar komórek w siatce dyskretyzacji parametrów.
 - Uśrednianie zawartości komórek.
 - Jakie lokalne maksima są wystarczająco maksymalne ☺.
 - Jak znajdować odcinki? Linie proste rzadko rozciągają się od krawędzi do krawędzi obrazu. Może się nawet zdarzyć, że to samo równanie prostej opisuje dwa (lub więcej) odcinki leżące w różnych częściach obrazu.

Transformata Hough'a

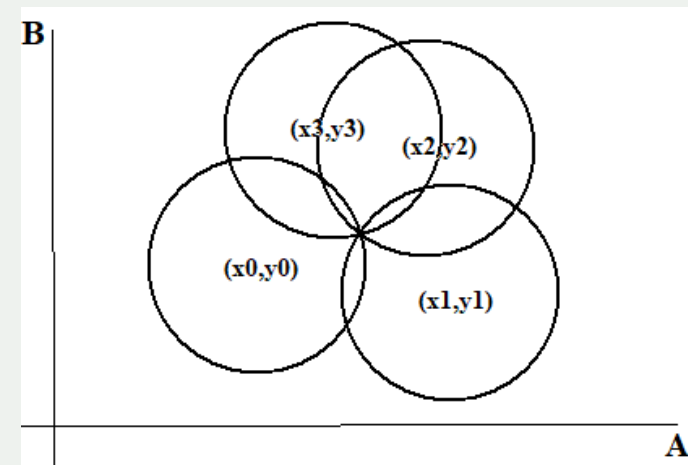
Detekcja innych linii w obrazach konturowych (praktycznym ograniczeniem jest jedynie liczba parametrów w równaniu linii, bo to determinuje wymiarowość przestrzeni Hough'a).

➤ Okręgi o stałym promieniu r_0

- Równanie okręgu $(x-a)^2 + (y-b)^2 = r_0^2$
- Przestrzeń Hough'a jest 2-wymiarowa $A \times B$.
- Pixel (x_0, y_0) generuje równanie $(x_0-a)^2 + (y_0-b)^2 = r_0^2$, a więc w przestrzeni Hough'a jest to też okrąg!
- Siatka dyskretyzacji jest skończona.

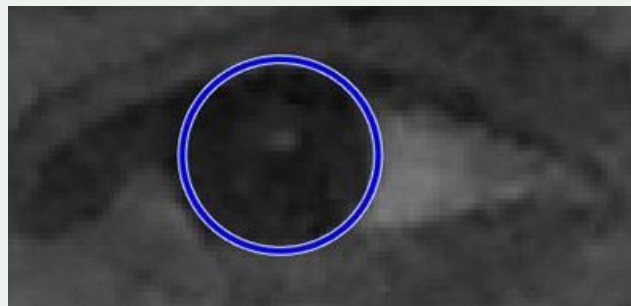
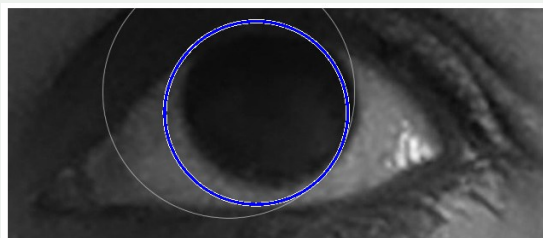
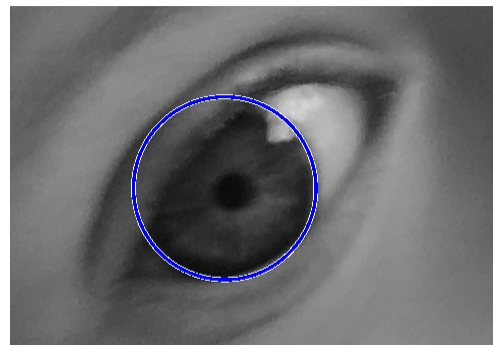
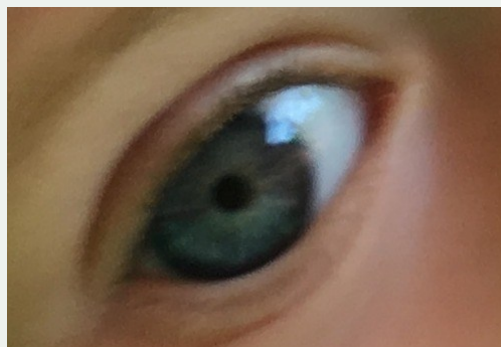
➤ Okręgi o dowolnym promieniu r

- Równanie okręgu $(x-a)^2 + (y-b)^2 = r^2$
- Przestrzeń Hough'a jest 3-wymiarowa $A \times B \times R$
- Pixel (x_0, y_0) generuje równanie $(x_0-a)^2 + (y_0-b)^2 = r^2$, a więc w przestrzeni Hough'a jest to równanie *powierzchni stożkowej*.
- Siatka dyskretyzacji jest skończona (ale 3-wymiarowa).



Transformata Hough'a

Przykład zastosowania kołowej transformaty Hough'a (w kontekście komunikacji z komputerem).

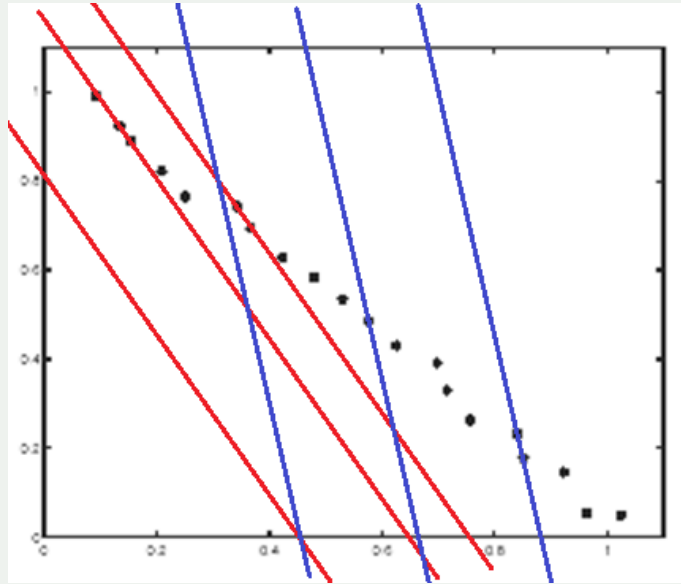


Transformata Hough'a $\Rightarrow$ transformata Radona

Nieco inna interpretacja transformaty Hough'a:

$$x \cos \theta + y \sin \theta = \rho \quad \Rightarrow \quad \begin{cases} x(t) = \rho \cos \theta + t \sin \theta \\ y(t) = \rho \sin \theta - t \cos \theta \end{cases} \quad -\infty < t < +\infty$$

równanie parametryczne



Dla każdej linii, odpowiadającej pewnym wartościom ρ i θ , „sumujemy” liczbę punktów konturowych leżących wzdłuż tej linii (w praktyce zawartych w wąskiej „rurce”, której szerokość zdeterminowana jest przez dyskretyzację parametru ρ w równaniu linii prostej) $x \cos \theta + y \sin \theta = \rho$.

Transformata Hough'a $\Rightarrow$ transformata Radona

Zatem, formalnie (zakładając, że $c(x,y)$ oznacza obraz konturowy z punktami konturowymi o wartości 1) transformatę Hough'a można zapisać (teoretycznie całkowanie, a w praktyce sumowania) jako:

$$HT(\rho, \theta) = \int_{-\infty}^{+\infty} c(\rho \cos \theta + t \sin \theta, \rho \sin \theta - t \cos \theta) dt \quad \begin{cases} x(t) = \rho \cos \theta + t \sin \theta \\ y(t) = \rho \sin \theta - t \cos \theta \end{cases}$$

Jeżeli jednak zamiast obrazu konturowego użyjemy dowolnego obrazu $i(x,y)$, to ten sam wzór (z taką samą interpretacją, tzn. „sumujemy” obraz wzdłuż odpowiednich linii) definiuje *transformatę Radona*.



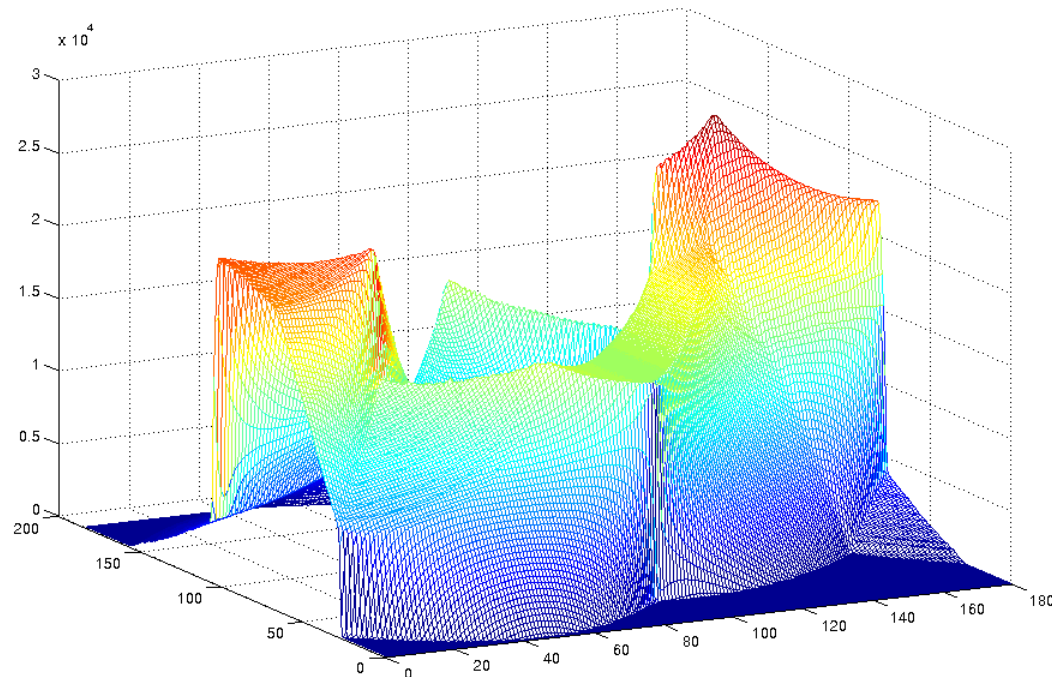
$$RT(\rho, \theta) = \int_{-\infty}^{+\infty} i(\rho \cos \theta + t \sin \theta, \rho \sin \theta - t \cos \theta) dt$$

Transformata Radona

Przykład:



obraz

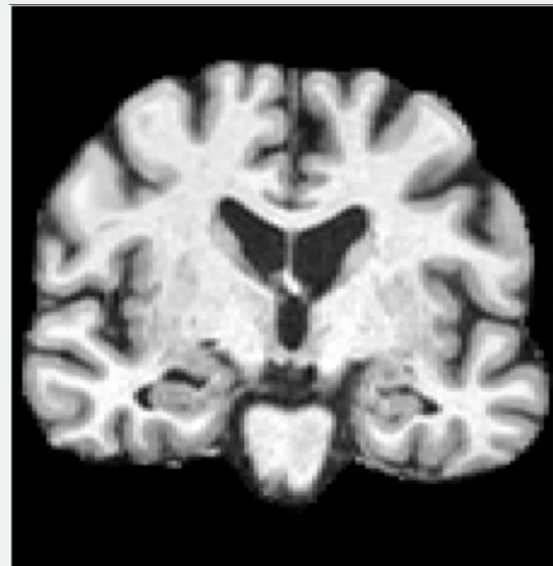


transformata Radona

Transformata Radona

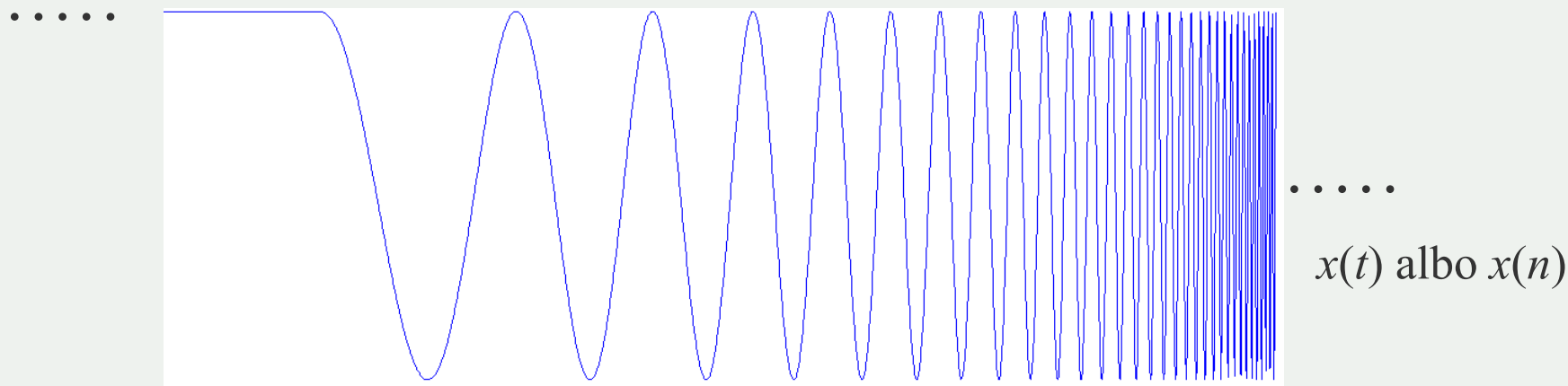
Tak więc wartość *transformaty Radona* dla wybranych (θ, ρ) oznacza po prostu zsumowanie („prześwietlenie”) obrazu wzdłuż linii o orientacji θ i o odległości ρ od środka układu współrzędnych. W praktyce, tak właśnie działa tomografia komputerowa, gdzie obracamy głowicę prześwietlając pacjenta wiązką równoległych linii (promieni) z różnych kierunków.

Ponieważ *transformata Radona* jest odwracalna oznacza to, że z takich „prześwietleń” uzyskanych pod różnymi kątami możemy odtworzyć (używając *odwrotnej transformaty Radona*) treść obrazu, czyli np. wewnętrzny przekrój ciała pacjenta.



Krótkookresowa transformata Fouriera

W przypadku długich sygnałów, globalne charakterystyki częstotliwościowe (transformata Fouriera) zazwyczaj nie opisują w użyteczny sposób treści takich sekwencji. Różne częstotliwości mogą występować w różnych fragmentach sygnałów/obrazów, a transformata Fouriera informuje nas jedynie o tym, że dana częstotliwość występuje w analizowanym sygnale z określoną amplitudą i fazą.

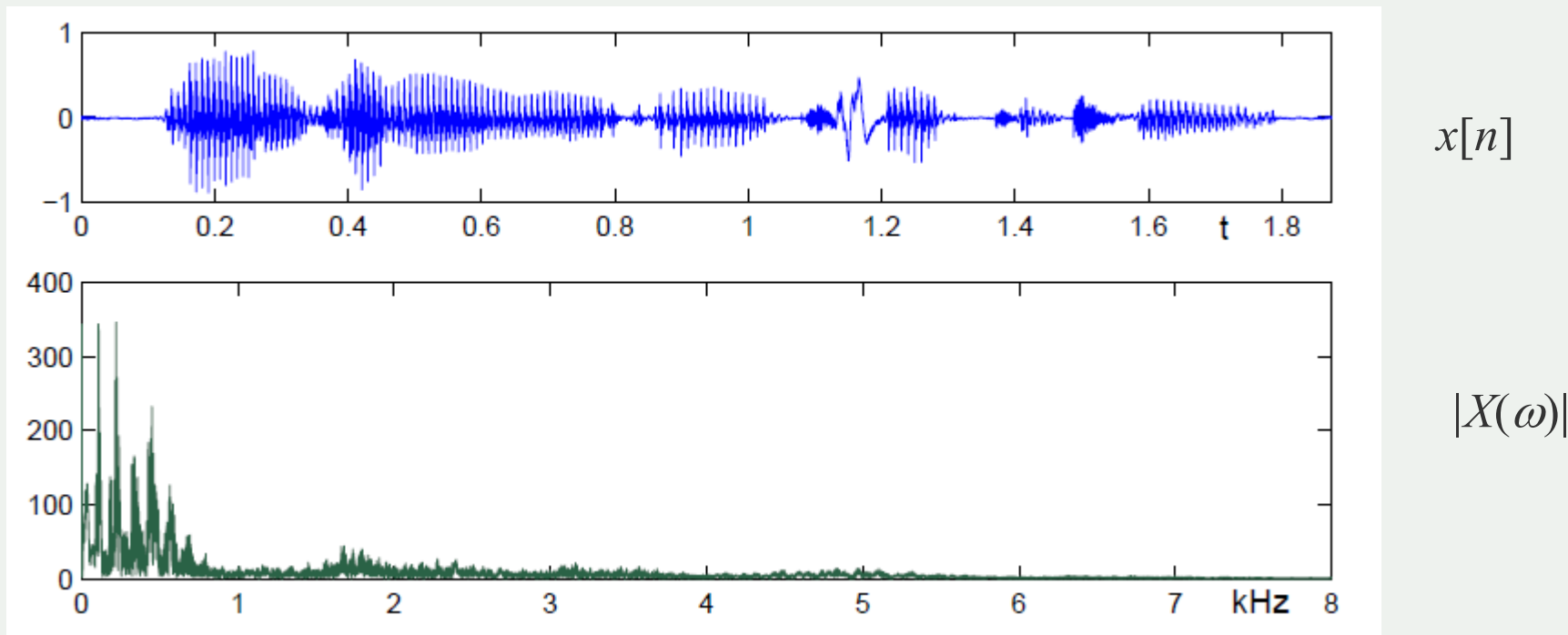


W takich sytuacjach korzystne jest użycie tzw. **krótkookresowej transformaty Fouriera** (ang. **STFT**, *short-term Fourier transform*).

Jest ona szczególnie przydatna przy analizie i przetwarzaniu sygnałów akustycznych (np. w głosowej komunikacji między człowiekiem i komputerem).

Krótkookresowa transformata Fouriera

Przykładowe dane akustyczne



Dla bardziej precyzyjnej interpretacji takich danych należy wiedzieć kiedy (w sensie współrzędnych czasowych) poszczególne częstotliwości pojawiają się w tym sygnale.

W tym celu wykorzystujemy *krótkookresową transformatę Fouriera*.

Krótkookresowa transformata Fouriera

Definicje transformaty *STFT* dla danych ciągłych $x(t)$ i zdyskretyzowanych $x[n]$:

$$\mathcal{FT}_s[x(t)] = X(\omega, \tau) = \int_{-\infty}^{+\infty} x(t)w(t-\tau)e^{-j\omega t}dt$$

gdzie $w(t)$ jest funkcją okna.

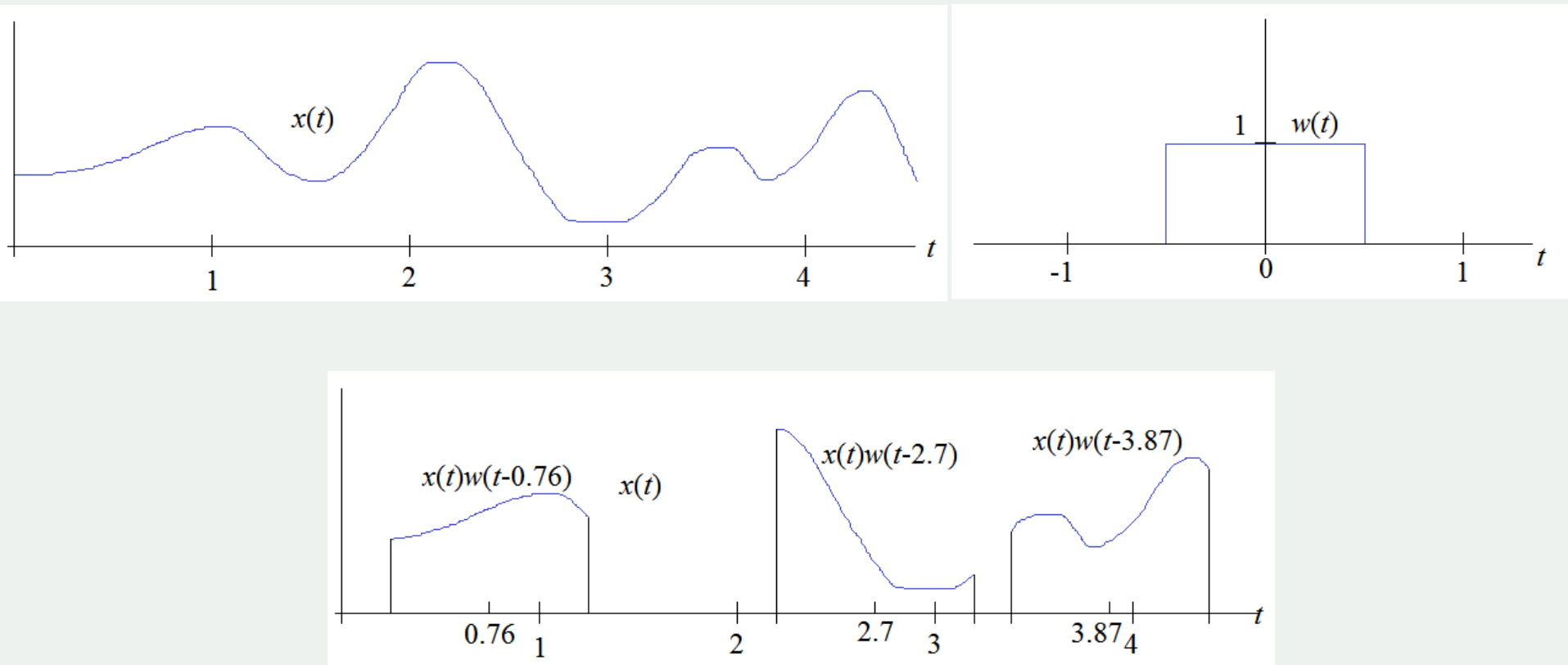
$$\mathcal{DTFT}_s\{x[n]\} = X(\omega, N) = \sum_{n=-\infty}^{+\infty} x[n]w[n-N]e^{-j\omega n}$$

gdzie $w[n]$ jest funkcją okna.

Przesuwająca się *funkcja okna* $w(t-\tau)$ lub $w[n-N]$ „wybiera” fragment danych w zależności od aktualnego położenie okna.

Tak więc *STFT* jest funkcją dwóch argumentów: *częstotliwości* oraz *położenia okna*, tzn. określa ona jakie częstotliwości (i z jakimi amplitudami/fazami) występują we fragmencie zdefiniowanym przez dane położenie okna.

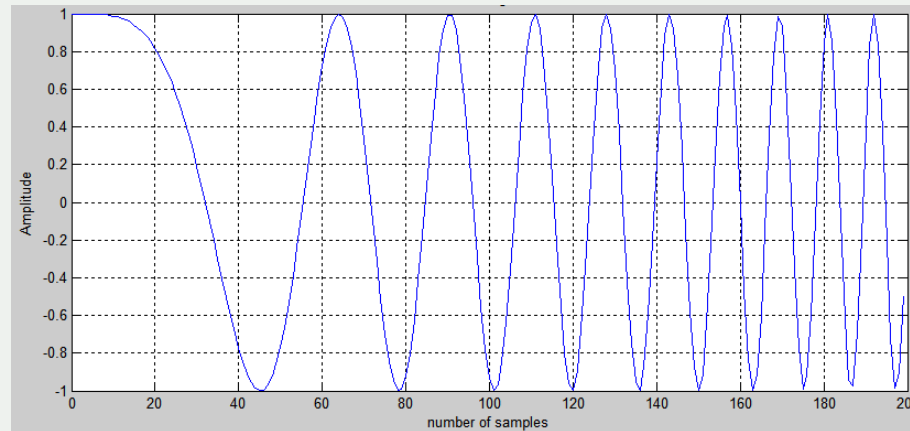
Krótkookresowa transformata Fouriera



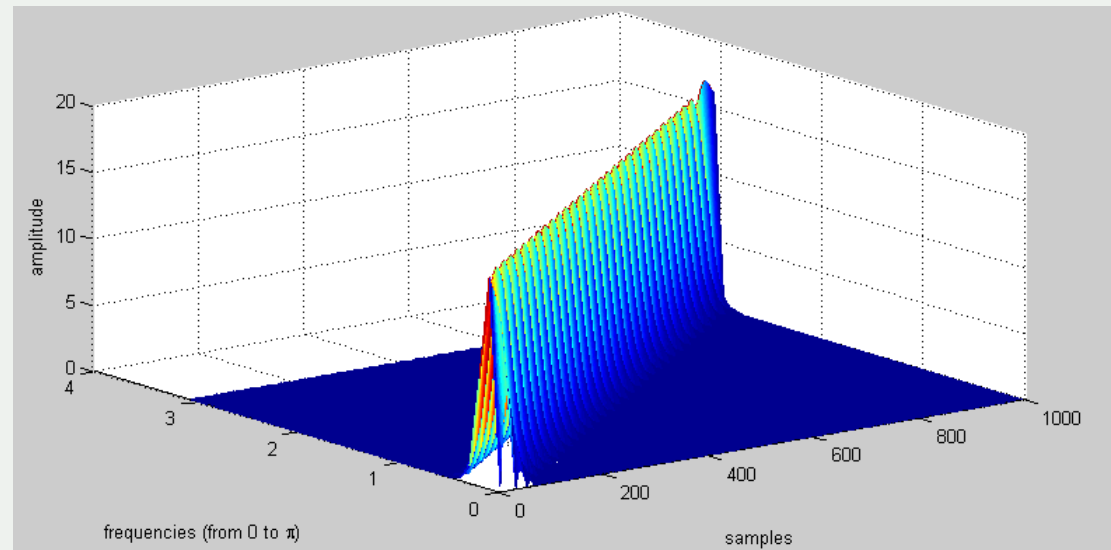
Krótkookresowa transformata Fouriera

Wykres pokazujący zmiany STFT (tzn. zmiany amplitudy) w czasie nazywamy *periodogramem*.

Przykładowe dane



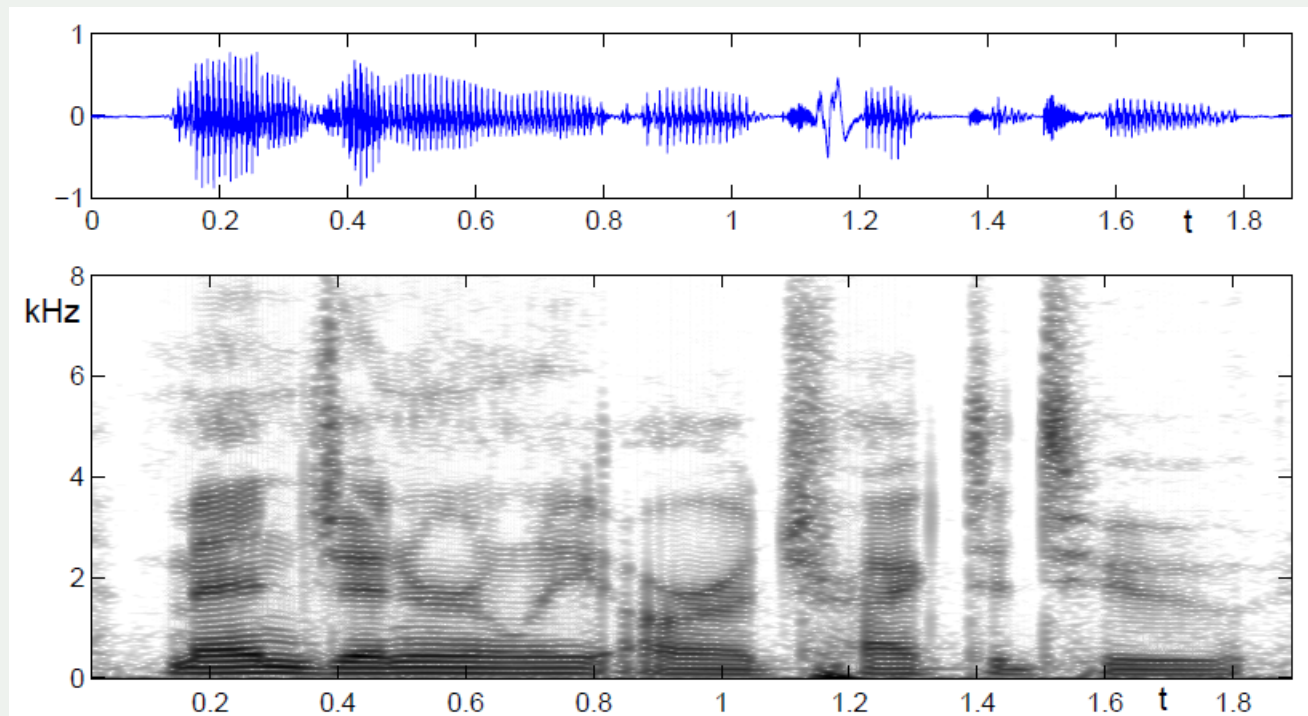
i ich *periodogram*.



Krótkookresowa transformata Fouriera

Z kolei *spektrogram* jest po prostu kwadratem *periodogramu*, tyle tylko, że zwykle reprezentowanym poziomami jasności). Jest to jedna z ważnych metod reprezentacji danych akustycznych na podstawie ich STFT.

Spektrogramy są szczególnie ważne na najniższych poziomach algorytmów analizy i syntezy dźwięku (np. rozpoznawanie mowy, konwersja *text-to-speech*, itd.).



Krótkookresowa transformata Fouriera

Intuicyjnie, funkcja okna powinna mieć kształt prostokątny. Niestety, taki kształt okna powoduje niepożądany efekt, tzw. *spectral leakage* (przeciek widma ☺ ?) .

Nieformalnie można to wyjaśnić w następujący sposób:

Sygnał $x[n]$ „obcięty” przez funkcję okna gwałtownie zaczyna się i kończy na obu końcach okna. Żeby „wygenerować” takie gwałtowne skoki, potrzebne są wysokie częstotliwości, które pojawiają się jako dodatkowe („przeciekające”) składowe o wysokich częstotliwościach w STFT danego fragmentu sygnału.

Dokładniejsza analiza tego problemu będzie pominięta.

Krótkookresowa transformata Fouriera

Przykład (dotyczący sygnałów akustycznych): Jak dobierać szerokość okna dla STFT stosowanej do analizy takich sygnałów?

Szerokość *okna* zależy od dwóch przeciwstawnych czynników:

1. Zakłada się, że w ramach całego okna charakterystyka częstotliwościowa sygnału powinna być (w przybliżeniu) stała, a więc **okno powinno być wystarczająco wąskie (krótkie)**.
2. Jeśli szerokość okna wynosi w , to częstotliwości niższe niż $1/w$ (w Hz) lub $2\pi/w$ (w radianach) są pomijane, bo okno jest za wąskie, żeby „zmieścić” w nim chociaż jeden pełny okres takich częstotliwości. Tak więc **okno powinno być wystarczająco szerokie (długie)**.

Człowiek odbiera sygnały akustyczne o częstotliwościach wyższych niż $\sim 20\text{Hz}$. Tak więc minimalna szerokość okna powinna wynosić ok.

$$\frac{1}{20[\text{Hz}]} = 0.05[\text{sek}] = 50[\text{milisekund}]$$

Krótkookresowa transformata Fouriera

Dla sygnałów cyfrowych szerokość *okna* mierzona jest liczbą próbek. Przy ustalonej czasowej długości okna, liczba próbek w oknie zależy od **częstotliwości próbkowania**.

W sygnałach akustycznych (gdzie maksymalne częstotliwości nie przekraczają $\sim 20\text{kHz}$) częstotliwość próbkowania powinna wynosić co najmniej 40kHz (czyli ok. 25 mikrosekund na jedną próbkę).

Ponieważ $\tau=50\text{ milisekund}$, w oknie o takim czasie trwania będzie $N = 2000$ próbek.

W praktycznych zastosowaniach, STFT sygnałów akustycznych jest najczęściej obliczana z użyciem okna o długości $N = 2048$ próbek.

Uwaga: Jeśli sygnały akustyczne próbkowane są z mniejszą częstotliwością (np. w telefonii cyfrowej) liczba próbek w oknie będzie odpowiednio mniejsza. Np. przy częstotliwości próbkowania 10kHz , w oknie o długości 50 milisekund będziemy mieli $N = 500$ próbek.

Transformacje falkowe (*wavelet transforms*)

Krótkookresowa transformacja Fouriera jest funkcją DWÓCH argumentów ω (częstotliwość) i N (położenie okna w analizowanej sekwencji) . Pierwszy określa jakie częstotliwości występują w sygnale, a drugi pokazuje w której części sygnału te częstotliwości występują.

Transformacje falkowe idą w podobnym kierunku, ale zamiast sinusoid o zróżnicowanej częstotliwości sygnały „rozkłada się” w inny sposób i na zupełnie inne funkcje bazowe.

Transformacje falkowe (*wavelet transforms*)

W *transformacjach falkowych* sygnały „rozkładamy” na funkcje o określonym kształcie (spełniające pewne wymagania matematyczne) które mogą być *przeskalowane* (w pewnym sensie odpowiada to zmianie częstotliwości) oraz *przesunięte* (w pewnym sensie odpowiada to przesuwającej się funkcji okna).

Zakładając pewien „wzorcowy” kształt tych funkcji $\psi(t)$, funkcje bazowe mają ogólną postać:

$$\frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right)$$

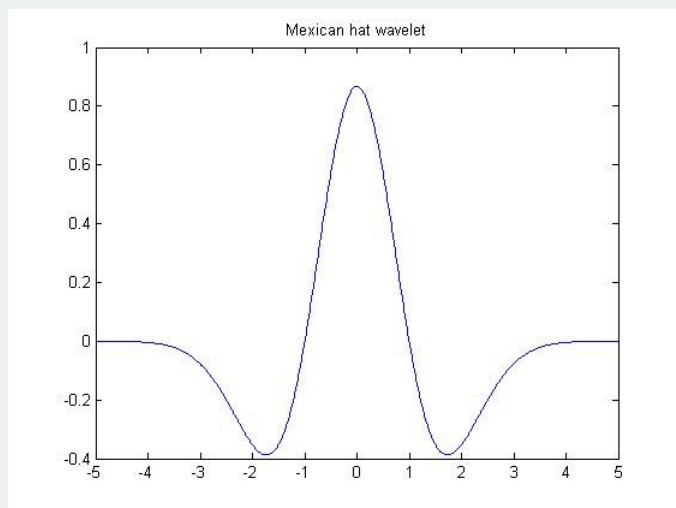
gdzie symbol b reprezentuje *przesunięcie* (opóźnienie) danej funkcji bazowej, a symbol a reprezentuje jej *skalę*.

Normalizacja amplitudy (pierwiastek z a) wymagana jest ze względów matematycznych.

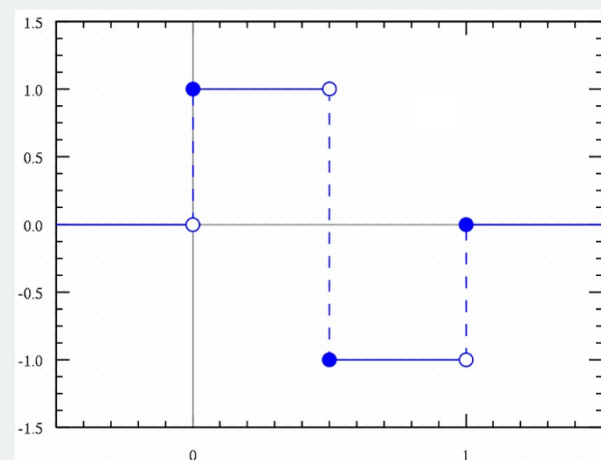
Transformacje falkowe (*wavelet transforms*)

Gdy $b = 0$ i $a = 1$, mamy „wzorcową” funkcję bazową $\psi(t)$, którą nazywamy *falką podstawową* (lub *falką matką*, ang. *mother wavelet*).

Funkcje ψ muszą spełniać pewne warunki, ale w sumie istnieje kilka najbardziej popularnych kształtów falek podstawowych definiujących odpowiednie transformacje falkowe, np.:



Falka matka “sombbrero”

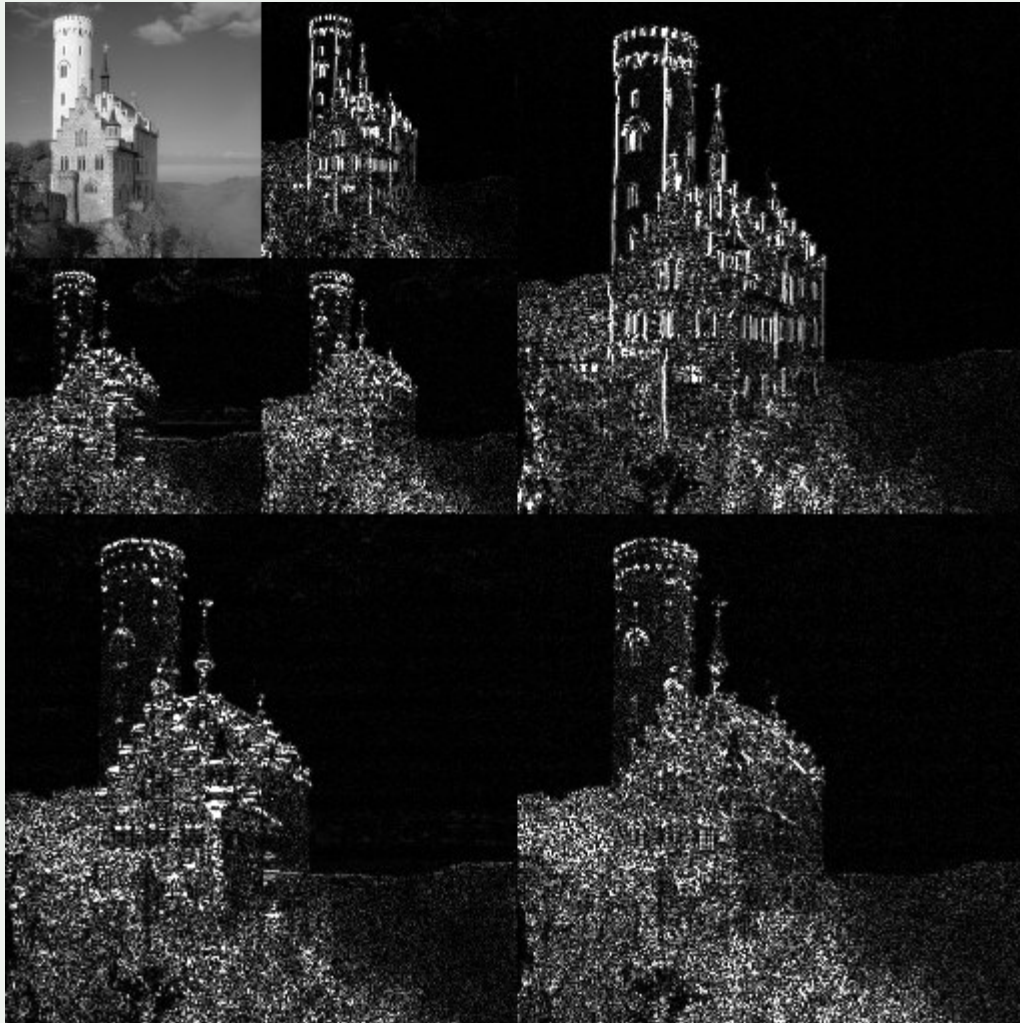


Falka matka Haara

Transformacje falkowe (*wavelet transforms*)

Przykład ilustrujący wykorzystanie transformacji falkowych dla danych 2D (obrazów).

$\times 2$

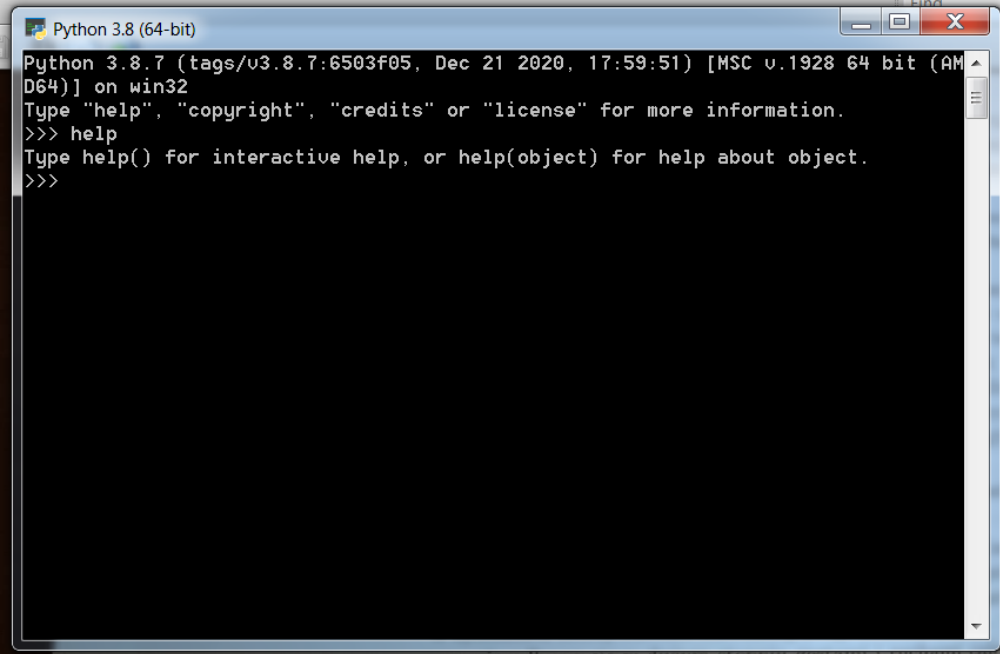


$\times\sqrt{2}$

Modele interakcji

Historycznie, modele interakcji między człowiekiem i komputerem ewoluowały w następujący sposób:

- **Brak interakcji** (wsadowy model działania komputera).
- **Interakcja tekstowa** (np. *command window*). Ograniczone/utrudnione możliwości interaktywnego przekazywania danych (zwłaszcza wyników). Najczęściej realizowany w postaci poleceń jednowierszowych (*command line interface*).



```
Python 3.8 (64-bit)
Python 3.8.7 (tags/v3.8.7:6503f05, Dec 21 2020, 17:59:51) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> help
Type help() for interactive help, or help(object) for help about object.
>>>
```

Modele interakcji

- **Interakcja oparta na ikonach** lub przyciskach (*icon-modal style* lub, w prostszej wersji, *key-modal style*). W tym modelu mieszczą się najróżniejsze warianty interakcji, ale wspólną ich cechą jest wybór spośród skończonej liczby aktualnie dostępnych opcji (ikony, klawisze funkcyjne, skróty klawiszowe, itp.).



Modele interakcji

➤ **Interakcja wykorzystująca manipulację obiektami.** Obejmuje kilka różnych modeli (zwykle współistniejących), w których użytkownik manipuluje obiektami graficznymi w celu wykonania wybranych operacji lub pobrania ich wyników. W ramach tego rodzaju interakcji można wyróżnić następujące kategorie:

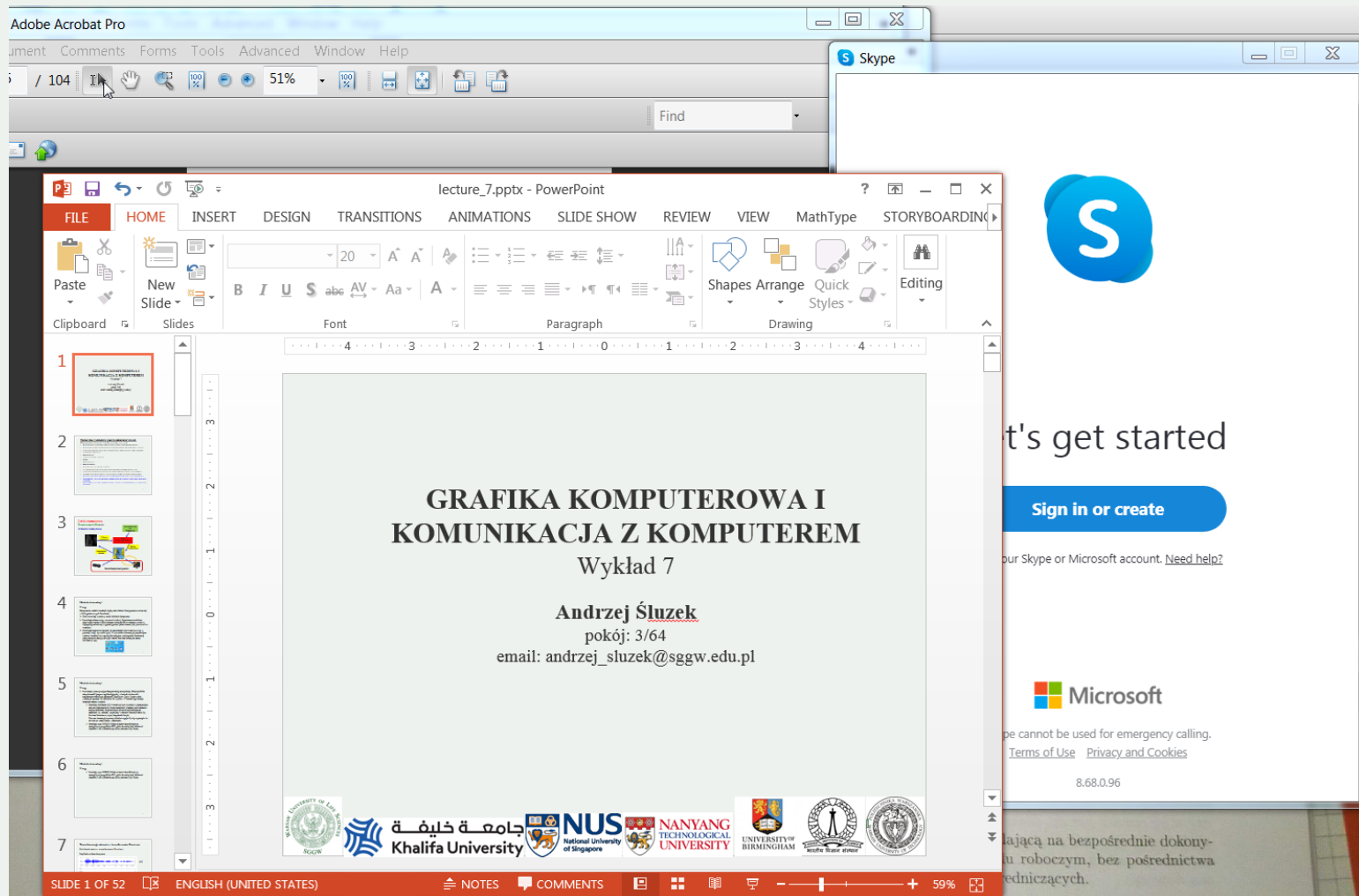
- ✓ **Interfejsy graficzne** (GUI – *graphical user interface*) wykorzystujące okna do przełączania się między zadaniami, przekazywania obiektów między zadaniami, skupiania się na aktualnie najważniejszych zadaniach, itp. Metoda „współpracy” z oknami może być różna (np. klawisze kierunkowe, mysz, bezpośredni dotyk).

Pierwsze komercyjne systemy okienkowe pojawiły się na początku lat 80-tych XX wieku (Xerox, Macintosh).

- ✓ **Interfejsy typu WIMP** (*Windows-Icons-Menu-Pointer*) są szczególnym przypadkiem GUI, gdzie do manipulacji obiektami (ikonami) i ich wybierania używa się pointerów (np. mysz).

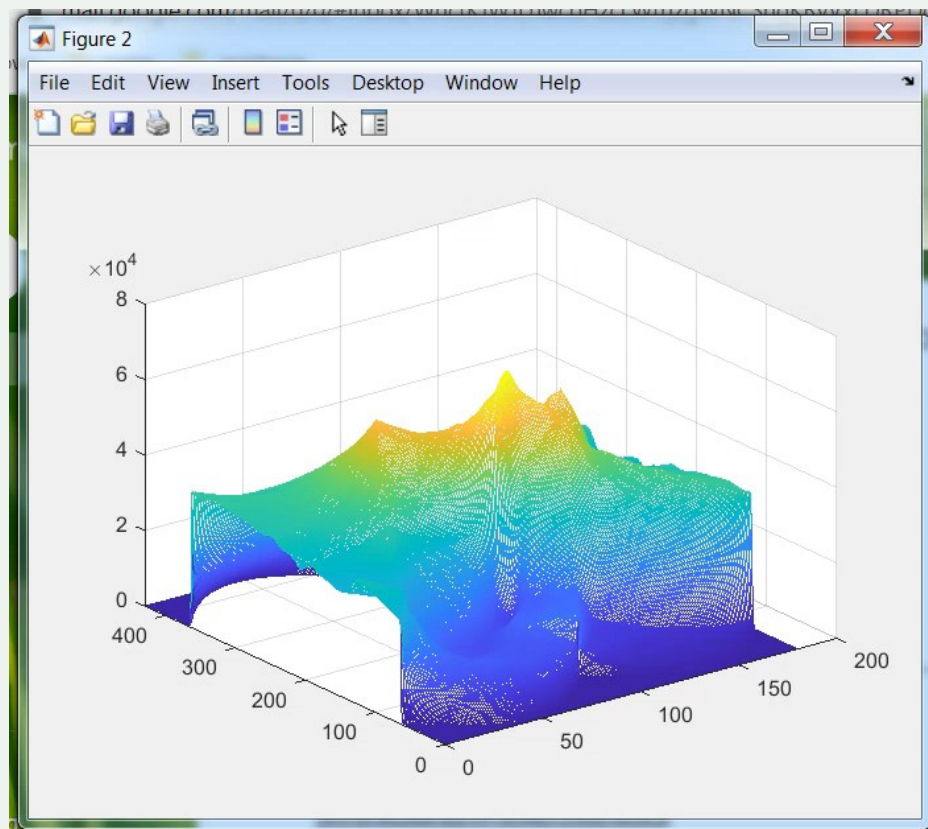
Modele interakcji

Przykład: GUI w połączeniu z WIMP



Modele interakcji

- ✓ **Interfejsy z użyciem manipulacji bezpośredniej** umożliwiające bezpośrednio obserwację wyników/danych (w tym 3D) bez konieczności otwierania/uruchamiania innej aplikacji.



Fizyczna metoda manipulacji (mysz, ekran dotykowy, joystick, itp.) ma drugorzędne znaczenie.

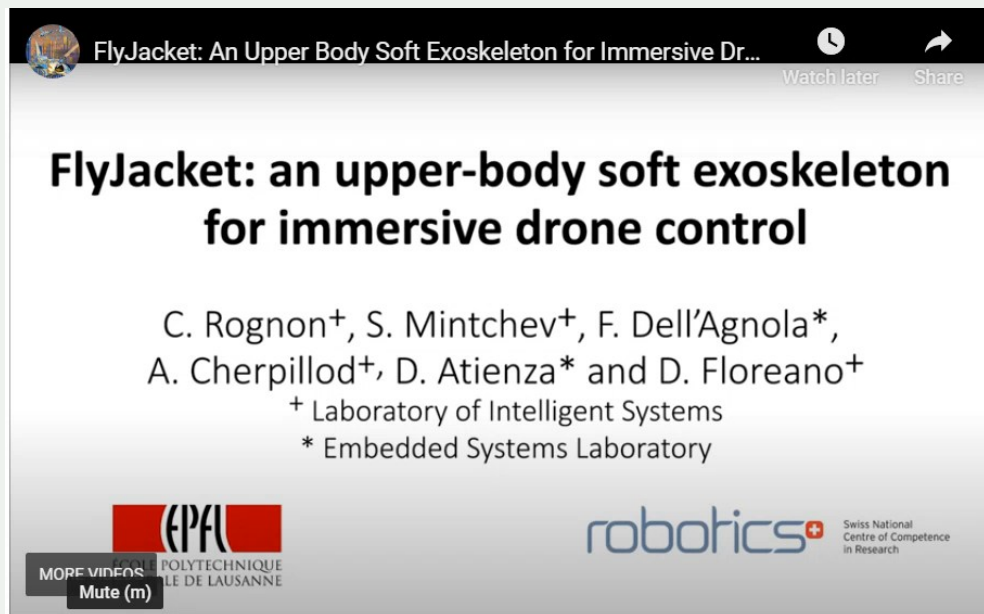
Modele interakcji

- **Interakcja wykorzystująca naturalne metody komunikacji.** Obejmuje kilka różnych technik, które w maksymalny sposób starają się odwoływać do metod komunikacji występujących w świecie rzeczywistym. Najbardziej typowe warianty takich interakcji to:
 - ✓ **Interakcja głosowa** występująca w wersji jednokierunkowej (synteza komunikatów głosowych przez komputer) lub w znacznie trudniejszej wersji dwukierunkowej (która dodatkowo wymaga systemu rozpoznawania mowy). Stosowana jest na przykład do interakcji z „inteligentnymi” robotami.



Modele interakcji

- ✓ **Interakcja kinetyczno-haptyczna** obejmująca różnego rodzaju „zanurzenia” w rzeczywistości wirtualnej i komunikowanie się z nią przez gestykulację (wymagany system wizyjny do analizy ruchów ciała, np. Kinect) bądź dodatkowe dołączenie systemów haptycznych (dotykowych) dających jedno- lub dwukierunkowe „odczuwanie” dotyku i innych fizycznych interakcji (wymagane są złożone systemy elektromechaniczne).



Modele interakcji

- ✓ **Interakcja „rozmyta”** (wykorzystująca tzw. *ubiquitous computing*, czyli „wszechobecne moce obliczeniowe”). Różnorodne czujniki/systemy rozmieszczone na ludzkim ciele, lub w pobliżu człowieka, umożliwiają wymianę informacji z systemami komputerowymi na poziomie porównywalnym z komunikacją międzyludzką (a nawet bardziej zaawansowanym, np. „czytanie” intencji użytkownika) .
- ✓ **Interakcja multipersonalna.** Zanurzenie wybranych modeli interakcji naturalnej nie jest ograniczone do pojedynczych użytkowników. Wieloosobowa interakcja z tą samą przestrzenią wirtualną (albo z rzeczywistością rozszerzoną, ang. *augmented reality*) umożliwia tworzenie interfejsów o nie w pełni jeszcze przewidywalnych możliwościach (a również potencjalnych zagrożeniach). Ten rodzaj interakcji jest podstawą systemów typu *metaverse*.

Zasady projektowania interfejsów graficznych

Zakres tematów

- Trochę o metodyce **UCD** (*user-centered design*).

User-centered design (UCD) jest iteracyjnym podejściem do procesu projektowania. W metodzie tej projektant skupia się na potrzebach użytkownika w każdej fazie procesu.

W szczególności zespoły projektowe włączają użytkowników w proces projektowania używając różnych metod tak, aby powstał produkt w pełni użyteczny i łatwy w eksploatacji.

- Wskazówki dotyczące wizualnych aspektów **interfejsów graficznych** (GUI – *graphical user interface*).

Nawiązanie do *heurystyk Nielsena* oraz *heurystyk Shneidermana* (zapewnienie funkcjonalności i ergonomii).

Zasady projektowania interfejsów graficznych

Podstawy metodyki UCD (*user-centered design*)

UCD to iteracyjny proces projektowania, gdzie w każdej fazie projektu na pierwszym planie są potrzeby użytkowników.

Zespół projektowy angażuje potencjalnych użytkowników w trakcie tworzenia produktu, wykorzystując w tym celu różne techniki i metody.

Każda iteracja procesu USD składa się z czterech (istotnie różnych od siebie) etapów:

- A. Zidentyfikowanie kontekstu w jakim użytkownicy będą wykorzystywać produkt.
- B. Zdefiniowanie wymagań.
- C. Zaprojektowanie rozwiązań.
- D. Porównanie (ewaluacja) jak produkt sprawdza się w kontekście zastosowań i czy spełnia wymagania tych zastosowań.

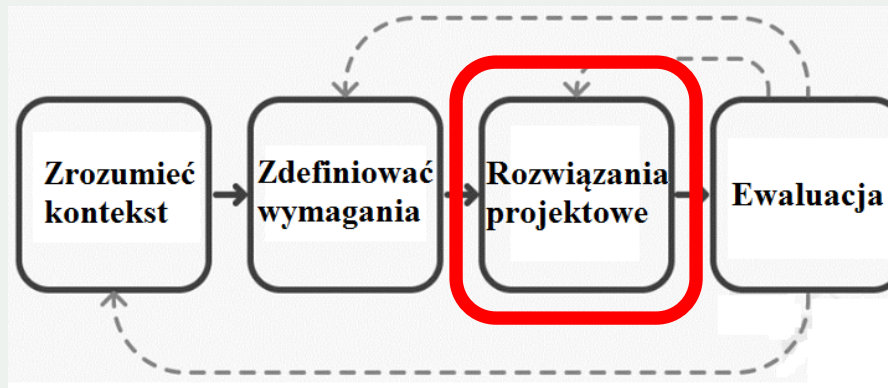
W punkcie (D) możemy wracać do (B) lub (C), jeśli zidentyfikowane zostaną niedociągnięcia.

A potem zaczynamy nową iterację, ponownie startując z punktu (A).

Zasady projektowania interfejsów graficznych

Podstawy metodyki **UCD** (*user-centered design*)

UCD to iteracyjny proces projektowania, gdzie w każdej fazie projektu na pierwszym planie są potrzeby użytkowników.



W UCD, projekty bazują na zrozumieniu *użytkowników, zadań i środowiska*, a głównym celem jest poznanie i odniesienie się do doświadczeń użytkowników we wszystkich trzech aspektach. Wymagać to może zaangażowania ekspertów z innych dziedzin niż zakres projektu.

Dwa najważniejsze punkty to:

- W każdym etapie powinni być zaangażowani potencjalni użytkownicy.
- Doświadczenia/oceny użytkowników powinny być gromadzone w wystarczająco długim/zróżnicowanym cyklu badań.

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- **Zastosowanie**
- **Funkcjonalność i ergonomia**
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Wymagania wynikające z charakteru zadania

Zastosowanie projektowanego systemu (typowe przykłady zagadnień)

- Szybkość interakcji (czas rzeczywisty?, ograniczony czas reakcji?, prezentacja wyników obliczeń?)
- Bezpieczeństwo danych (nieistotne?, krytyczny wymóg?, na ile istotna jest bezbłędna reakcja użytkownika?)

Funkcjonalność i ergonomia (często oceniana wg tzw. heurystyk Nielsena, 1990-94)

- status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)
- swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (np. przewidywalny czas reakcji, dostępność do całej „przestrzeni roboczej”)
- spójność (te same metody i standardy we wszystkich fragmentach/etapach)
- ułatwione rozpoznawanie (np. ikon) zamiast ciągłego przypominania ich znaczenia
- przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom
- właściwa terminologia (nie wykraczająca poza wiedzę użytkownika)
- elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)
- estetyka i „oszczędność” graficzna
- diagnostyka i korekcja błędów (czytelny opis błędu, operacja *undo*)
- dokumentacja (łatwo dostępna pomoc na żądanie)

Zasady projektowania interfejsów (graficznych)

Wymagania wynikające z charakteru zadania

Funkcjonalność i ergonomia (alternatywne tzw. złote zasady Shneidermana, 1986)

- Dążenie do spójności.
- Umożliwienie doświadczonym użytkownikom korzystanie ze skrótów.
- Maksymalizacja informacji zwrotnej.
- Interakcja podkreślająca początek, trwanie i zakończenie realizowanych zadań.
- Prosta obsługa błędów (i minimalizacja ryzyka poważnych błędów).
- Łatwe przywracanie wyników sprzed wykonanych czynności.
- Dawanie poczucia, że to użytkownik kontroluje system.
- Minimalizacja obciążenie pamięci krótkotrwałej.

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- **Zgodność ze standardami (formalnymi i nieformalnymi)**

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Wymagania wynikające z charakteru zadania

Zgodność ze standardami (formalnymi i nieformalnymi)

- zgodność z normami międzynarodowymi określającymi wymogi wobec produktów informatycznych, np. norma ISO/IEC 9126:
 - zrozumiałość (ang. *understandability*)
 - użyteczność (ang. *usability compliance*)
 - możliwość nauczania się (ang. *learnability*)
 - łatwość obsługi (ang. *operability*)
 - atrakcyjność (ang. *attractiveness*)
- lub norma ISO 9241:
 - skuteczność (ang. *effectiveness*)
 - efektywność (ang. *efficiency*)
 - satysfakcja (ang. *satisfaction*)
- zgodność ze standardami środowiska systemowego

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- **Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)**
- **Możliwości percepcyjne i pamięciowe użytkownika**
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Profil użytkownika

Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)

- np. profilowanie użytkowników wg tzw. *modelu Rasmussena*
 - działanie oparte na odruchach (ang. *skill-based*) charakteryzujące dobrze wyszkolonych operatorów działających w znanej przestrzeni informatycznej
 - działanie oparte na regułach (ang. *rule-based*) z wykorzystaniem prostych kryteriów typu *if.... then...* (gorzej wyszkoleni użytkownicy)
 - działanie oparte na wiedzy (ang. *knowledge-based*) zakładające znaczną kreatywność i elastyczność w podejmowaniu decyzji

Zasady projektowania interfejsów (graficznych)

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

- modele pamięci i percepcji
 - pamięć krótko- i długoterminowa (zmiany w zależności od wieku i stanu umysłowości)
 - percepcja, np. rola sygnałów dźwiękowych (natężenie, barwa, częstotliwość), dobór formy komunikatów wizualnych (więcej później)
 - „nasycenie” zmysłów i uwagi (problem zmęczenia, dekoncentracji lub znudzenia)
 - ograniczona precyzja ruchów (błędy przy manipulacji kursorem), tzw. prawo Fittsa

Zasady projektowania interfejsów (graficznych)

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

„Nasycenie” zmysłów (zasada magicznej siódemki).

Zasada ta opisuje ustalony empirycznie (i niekoniecznie prawdziwy w przypadku każdej osoby) fakt, że przy pojawieniu się więcej niż SIEDMIU niezależnych elementów (np. obiektów graficznych) następuje „nasycenie” zmysłów i analiza takiej informacji trwa znacznie dłużej.

Przykład „nasycenia”:

NBPPKOPZUNFZ

Ten sam przykład zrealizowany prawidłowo:

NBP PKO PZU NFZ

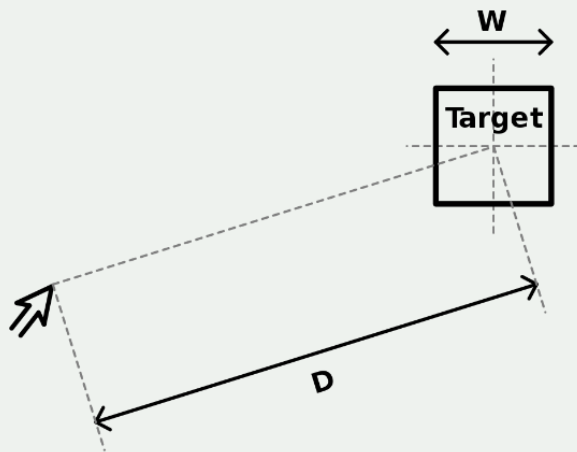
Zasady projektowania interfejsów (graficznych)

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

Prawo Fittsa to próba skwantyfikowania ograniczonych możliwości percepcyjno-ruchowych przeciętnego użytkownika manipulującego kursorem (np. na ekranie).

Prawo to postuluje, że szybkie i zarazem precyzyjne ruchy nie są jednocześnie możliwe i następująca zależność aproksymuje czas potrzebny do „trafienia” w obiekt:



$$t = x + y \log_2 \left(\frac{D}{W} + 1 \right)$$

gdzie x i y to stałe wyznaczone eksperymentalnie

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- **Charakterystyka nawyków i typowych błędów**

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Profil użytkownika

Charakterystyka nawyków i typowych błędów

- zachowania rutynowe (działanie „na pamięć” w niewłaściwych sytuacjach)
- błędy na skutek wysokiego poziomu stresu w sytuacjach krytycznych
- podzielność uwagi (konieczna lub nawykowa)
- potencjalna sprzeczność kryteriów decyzyjnych
- przybliżony model nabieranie wprawy, tzw. potęgowe prawo (t_1 , to czas pierwszej próby)

$$t_n = t_0 + t_1 n^{-\lambda}$$

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- **Sposoby pozyskiwania i przekazywania informacji**
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Realizacja projektu

Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Polecenia wierszowe (interakcja tekstowa)
- Pytanie-odpowieź
(do you like cheese? Y/N ... You said YES. Are you sure? Y/N ...)
- Przyciski funkcyjne



- Formularze (głównie zastosowania biznesowe)

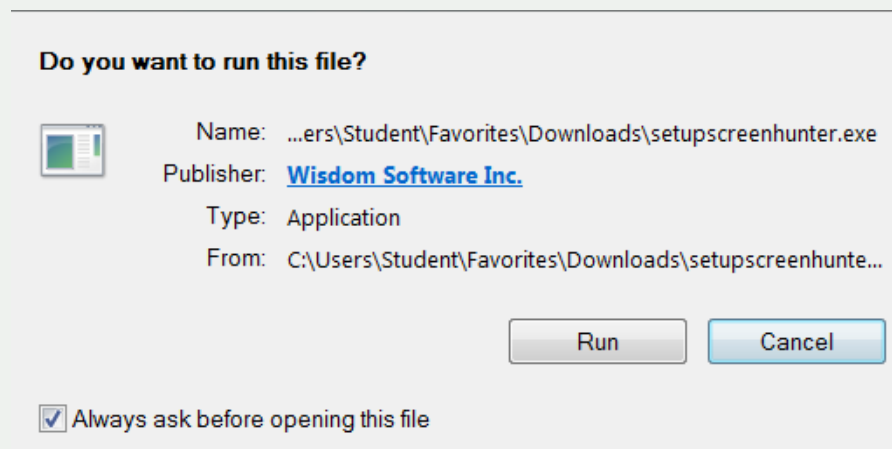
Zasady projektowania interfejsów (graficznych)

Realizacja projektu

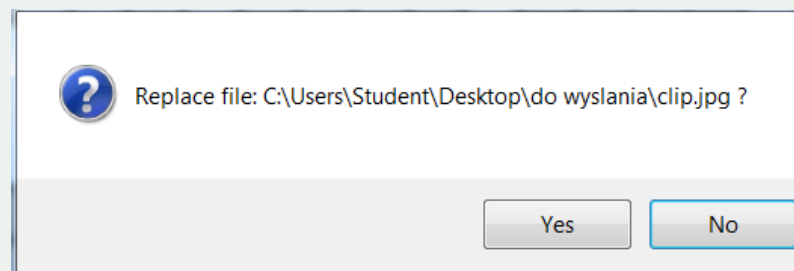
Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Okna dialogowe



- Komunikaty informacyjne



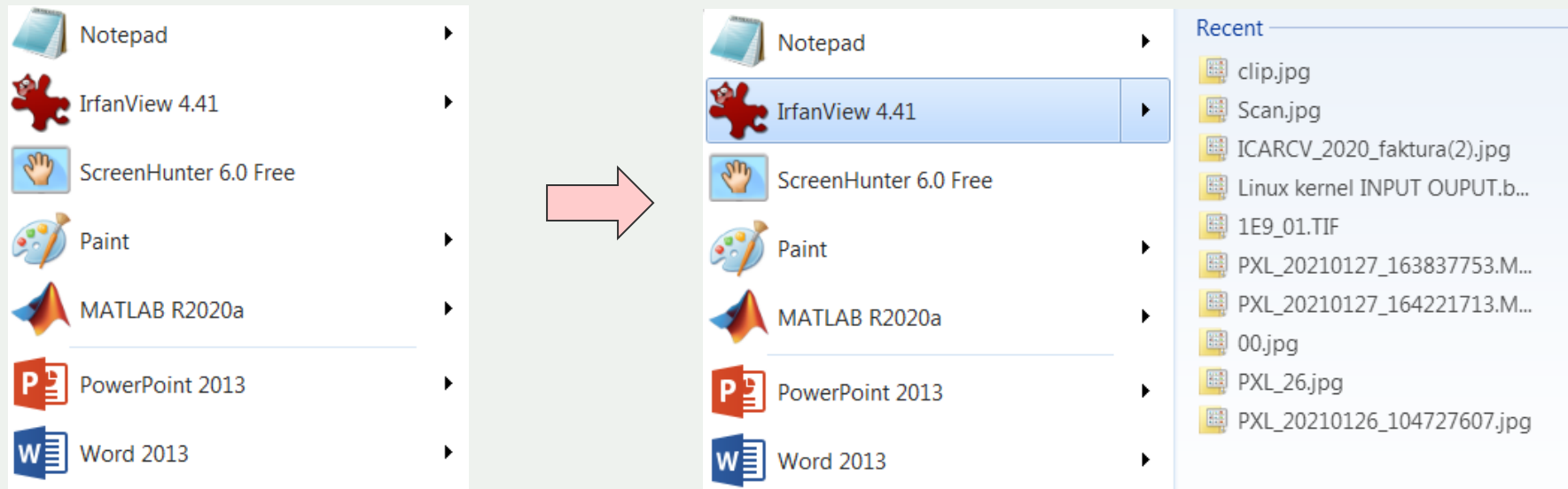
Zasady projektowania interfejsów (graficznych)

Realizacja projektu

Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Menu (w tym menu hierarchiczne, kaskadowe, wstążkowe, palety, zakładki, itp.)



Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- **Wykorzystanie istniejących rozwiązań**
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów (graficznych)

Realizacja projektu

Wykorzystanie istniejących rozwiązań i wzorców

Rola metafory (symbolicznej reprezentacji informacji) jest bardzo istotna, gdyż w możliwie najbardziej zwarty sposób powinna przekazywać niezbędną informację.

Metafory (np. forma graficzna ikon) powinna spełniać następujące wymagania:

- Czytelna znaczeniowo i graficznie (elementy wiedzy o kulturze potencjalnych użytkowników).
- Oszczędna (odwoływać się tylko do najważniejszych własności reprezentowanych obiektów).
- Pozbawiona nieistotnych „ozdobników”.
- Reprezentuje tylko obiekty/własności niezbędne dla wykonywanego zadania.
- Spójna znaczeniowo (w systematyczny sposób reprezentuje podobne obiekty w całym systemie). Być może należy korzystać z istniejących metafor.
- Nie sprawia wrażenia przestarzałej (np. taśma magnetofonowa jako metafora nagrywania dźwięku).
- Przyciąga uwagę użytkownika (więcej na kolejnych slajdach).

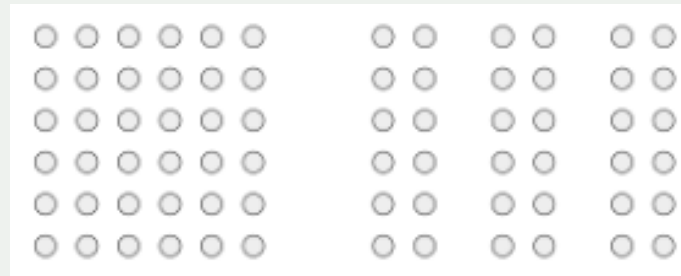
Zasady projektowania interfejsów graficznych

Podstawy percepcji wizualnej

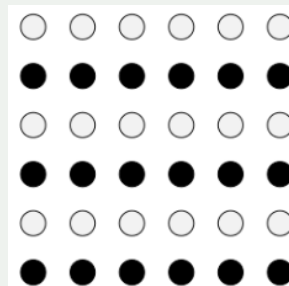
Wykorzystanie tzw. *praw (zasad) Gestalt* sformułowanych przez szkołę psychologii Gestalt na początku 20 wieku.

Psychologia Gestalt twierdzi, że prawa te istnieją, gdyż mózg ludzki ma naturalną tendencję do postrzegania wzorców, które można sklasyfikować na podstawie pięciu (sześciu?, siedmiu?) właściwości:

- Bliskość



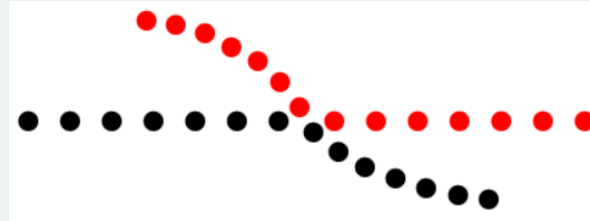
- Podobieństwo



Zasady projektowania interfejsów graficznych

Podstawy percepcji wizualnej

- Ciągłość (kontynuacja)



- Domknięcie (spójność percepcyjna)



- Symetria



Zasady projektowania interfejsów graficznych

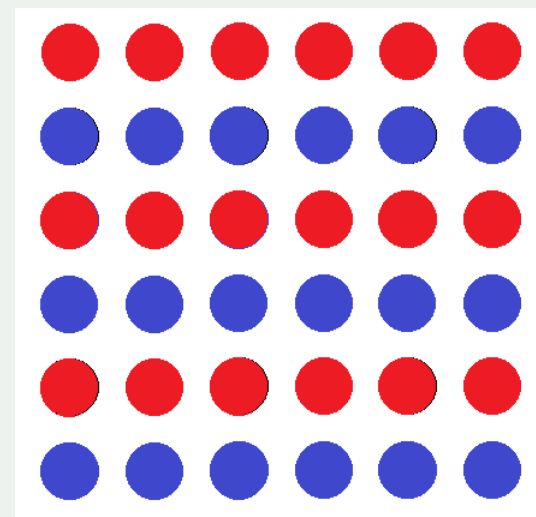
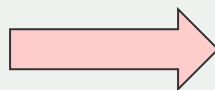
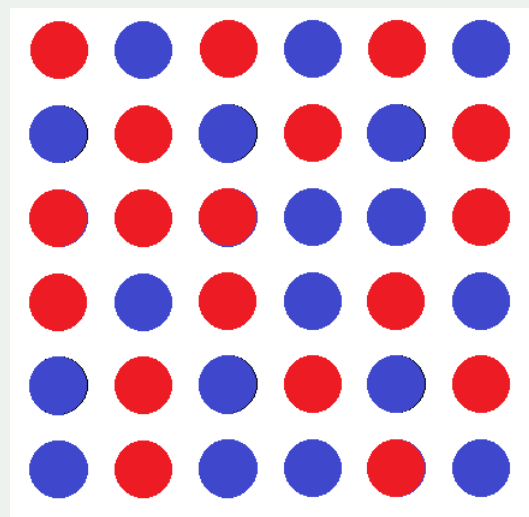
Podstawy percepcji wizualnej (przykład)

Używając tylko dwóch poniższych rodzajów obiektów wizualnych,

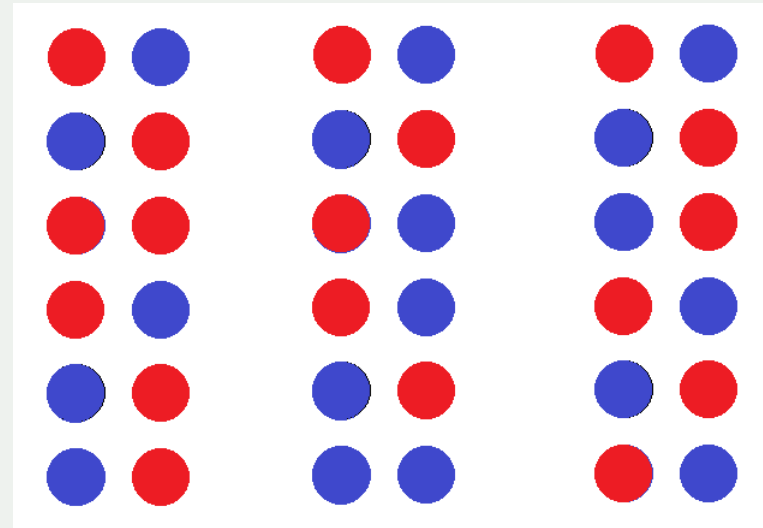
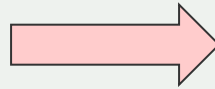
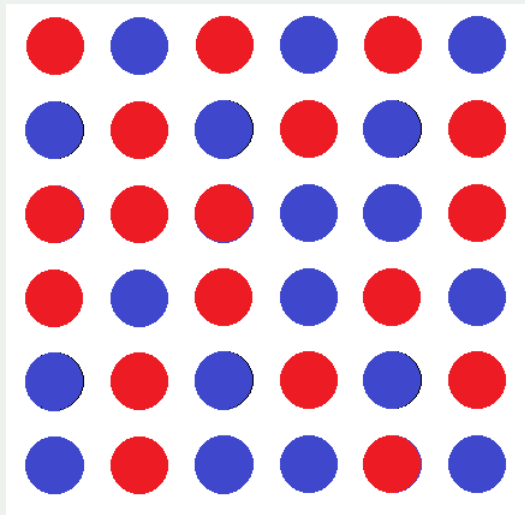


zbuduj z nich przykładowe rysunki (co najmniej po jednym do każdej zasady) ilustrujące jak każda z zasad Gestalt „pomaga” nam dostrzec w tych rysunkach oddzielne fragmenty.

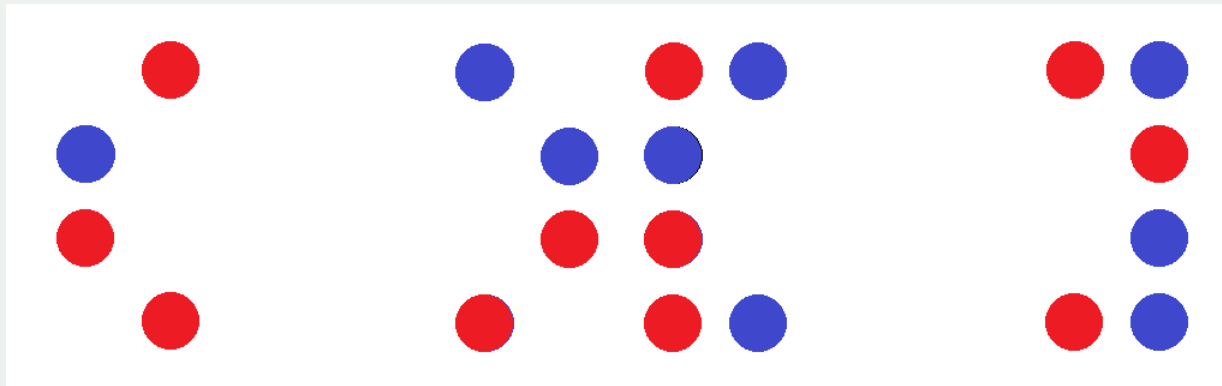
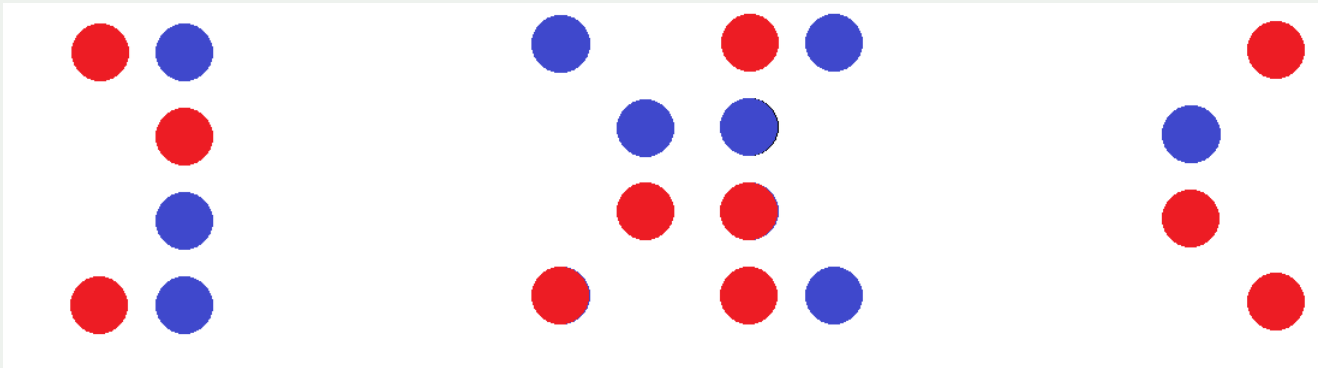
■ Podobieństwo



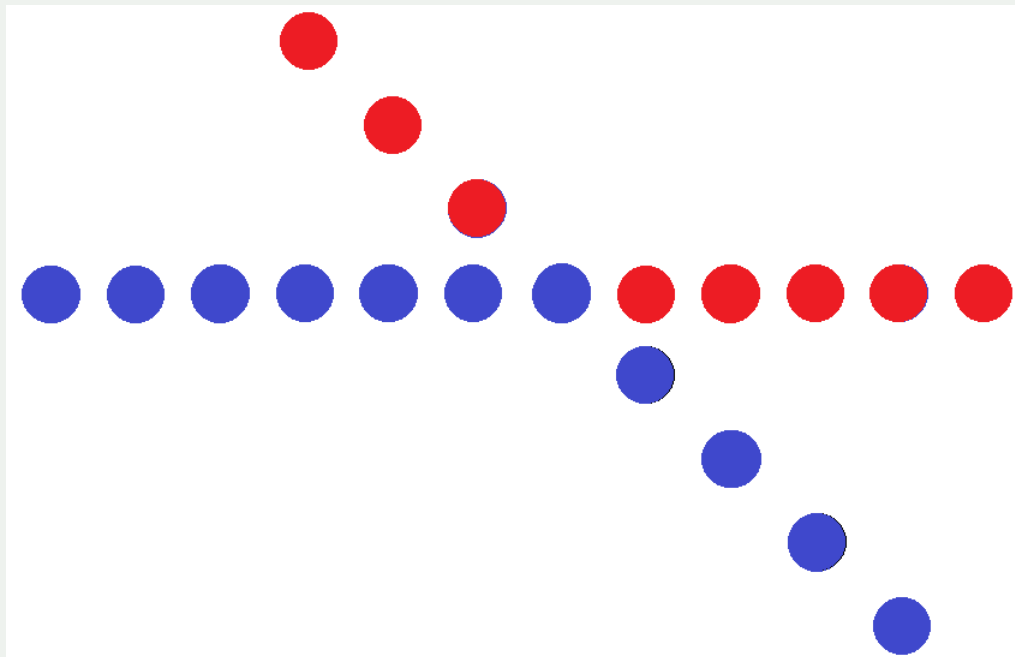
- Bliskość



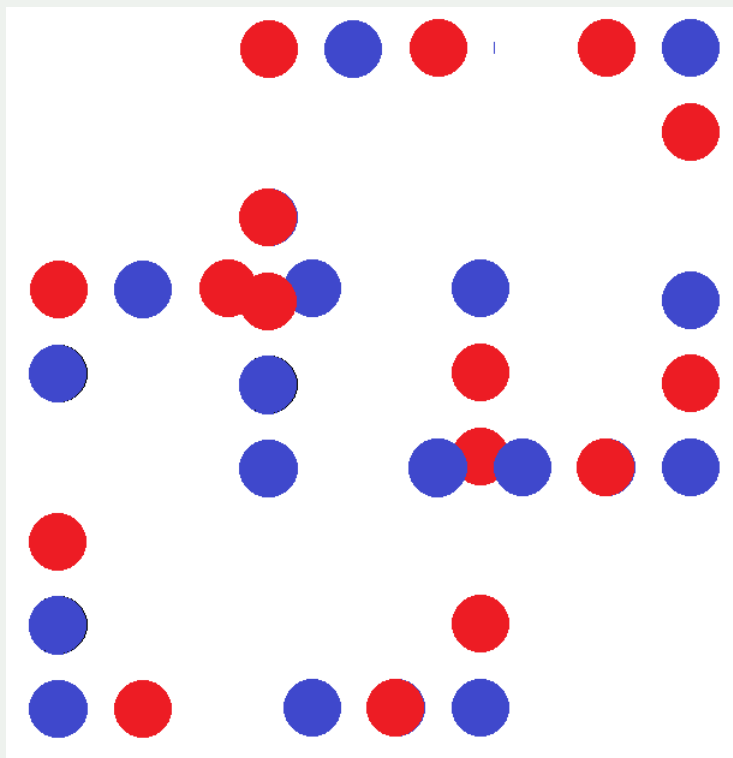
- Symetria



- Ciągłość (kontynuacja)



- Domknięcie (spójność percepcyjna)



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

Funkcjonalność i ergonomia interfejsów (graficznych) są często oceniane wg tzw. heurystyk Nielsena, 1990-94)

1. status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)
2. swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (np. przewidywalny czas reakcji, dostępność do całej „przestrzeni roboczej”)
3. spójność (te same metody i standardy we wszystkich fragmentach/etapach)
4. ułatwione rozpoznawanie (np. ikon) zamiast ciągłego przypominania ich znaczenia
5. przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom
6. właściwa terminologia (nie wykraczająca poza wiedzę użytkownika)
7. elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)
8. estetyka i „oszczędność” graficzna
9. diagnostyka i korekcja błędów (czytelny opis błędów, operacja *undo*)
10. dokumentacja (łatwo dostępna pomoc na żądanie)

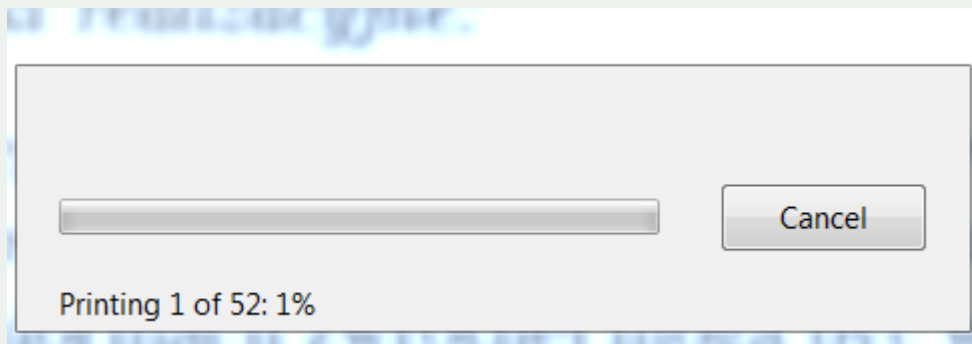
Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

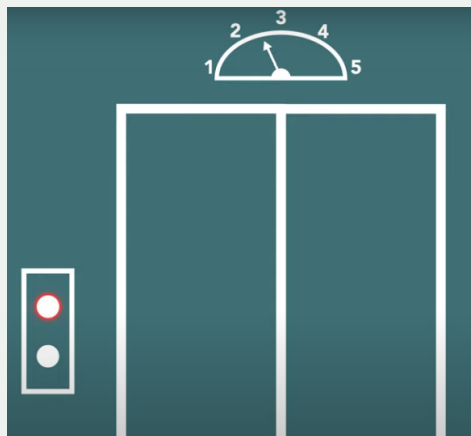
1. Status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)

Na przykład:

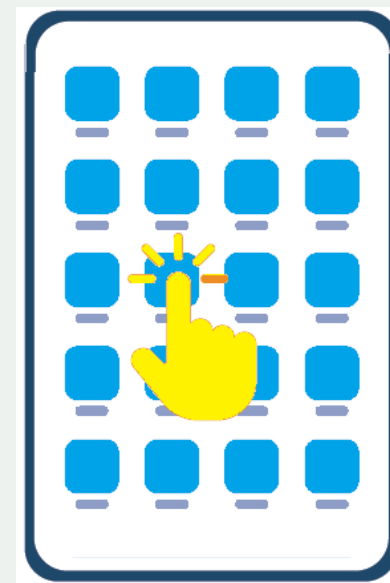
- Informacja zwrotna dostępna w najszerszym zakresie:



Pasek



Klepsydra



Dźwięk lub
podświetlenie

Zasady projektowania interfejsów graficznych

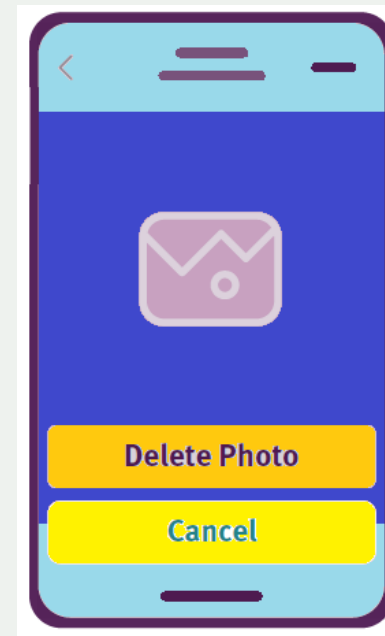
Wykorzystanie heurystyk Nielsena

2. Swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (dostępność całej „przestrzeni roboczej”, wyjścia awaryjne, itp.)

Na przykład:



Operacje *undo* i *redo* (tutaj dość źle zrealizowane)



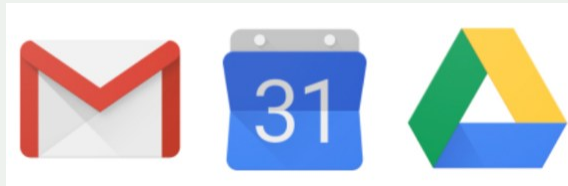
Dostępność przestrzeni roboczej

Zasady projektowania interfejsów graficznych

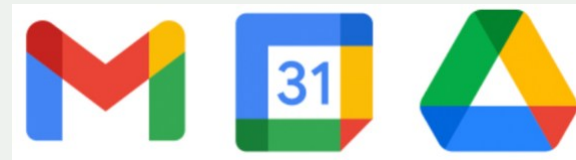
Wykorzystanie heurystyk Nielsena

3. Spójność (te same metody i standardy we wszystkich fragmentach/etapach)

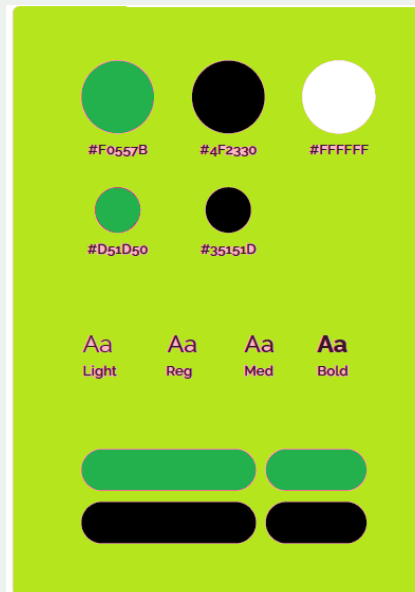
Na przykład:



Tak sobie



Dobrze



Kolor
Symbol
Czcionka

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

4. Ułatwione rozpoznawanie zamiast ciągłego przypominania znaczenia

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

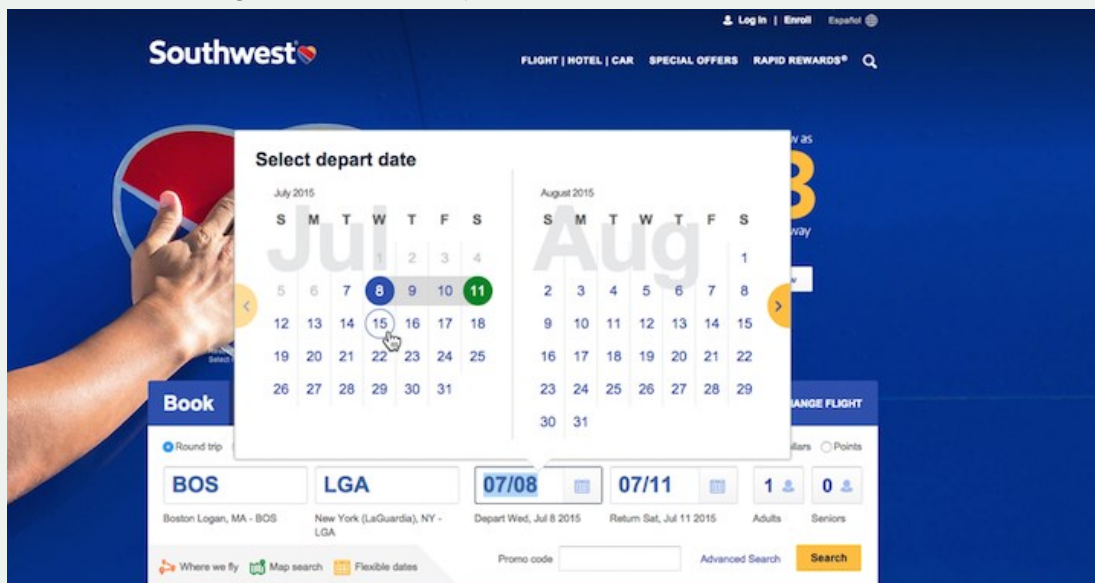
5. Przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom

Są dwa rodzaje błędów:

1. Pomyłki, ang. *slips* (popętnione nieświadomie).
2. Błędy, ang. *mistakes* (popętnione świadomie, np. niezrozumienie modelu)

Najbardziej „kosztowne” błędy mają najwyższy priorytet.

Pomyłki głównie w formie przypominania (tym wyraźniej, im potencjalnie „kosztowniejsze” skutki).



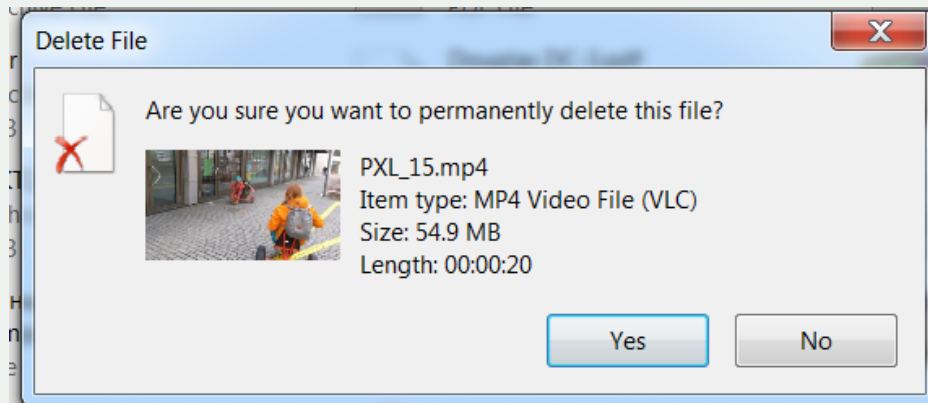
<https://www.nngroup.com/articles/slips/>

Zasady projektowania interfejsów graficznych

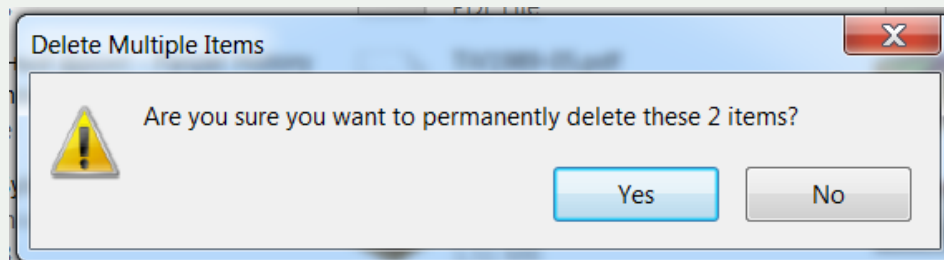
Wykorzystanie heurystyk Nielsena

5. Przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom

Potencjalne błędy (zwłaszcza błędy krytyczne) przedstawiane w postaci wyraźnych ostrzeżeń.



Dobre rozwiązanie



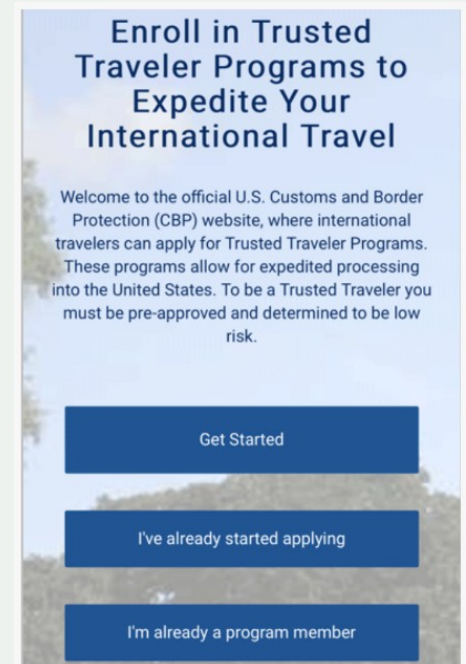
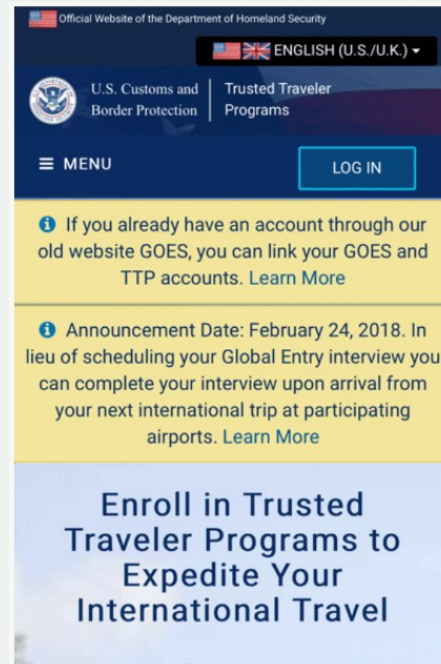
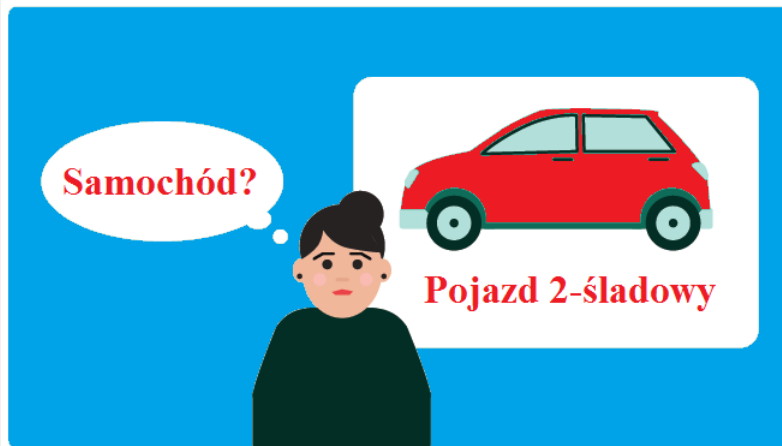
Gorsze rozwiązanie

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

6. Właściwa terminologia (zgodna ze „światem realnym”, nie wykraczająca poza wiedzę użytkownika)

Na przykład:



<https://www.nngroup.com/articles/match-system-real-world/>

Złe rozwiązanie i propozycja lepszej grafiki.

Zasady projektowania interfejsów graficznych

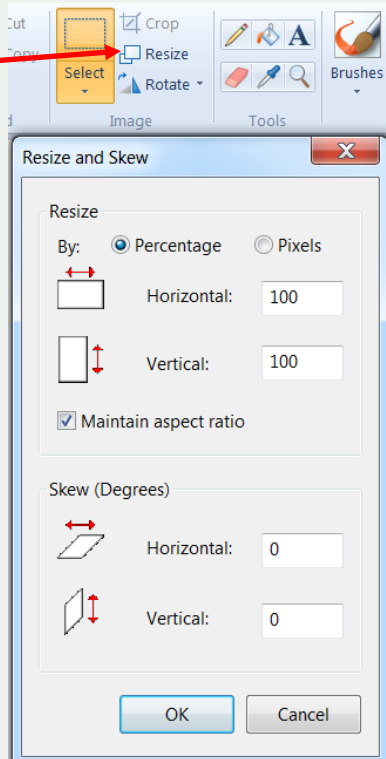
Wykorzystanie heurystyk Nielsena

7. Elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)

Oznacza zwykle *rezygnację* z pewnych elementów wizualizacji (zaawansowany użytkownik nie musi widzieć, żeby wiedzieć co zrobić).

Na przykład:

Zamiast wybierać
wciskamy CTRL+W



UWAGI:

„Kastomizacja” nie jest zalecanym rozwiązaniem, ponieważ większości użytkowników nie będzie się chciało tego robić,

Preferowana jest *personalizacja*, tzn. system automatycznie robi „kastomizację” na podstawie obserwacji zachowań użytkownika, ale....

personalizacja jest trudniejsze do realizacji i może być źródłem frustracji, jeśli zrobiona jest źle.

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

8. Estetyka i „oszczędność” graficzna

Na przykład:

ESTETYKA

ESTETYKA

Forma graficzna powinna być skupiona wokół głównego przekazu.

Elementy graficzne powinny wspierać podstawowe zadanie danego interfejsu, a nie rozpraszać uwagę użytkownika.

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

9. Diagnostyka i korekcja błędów (czytelny opis błędu, operacja *undo*, itd.)

Diagnostyka powinna wykryć każdy błąd. Najgorszy komunikat o błędzie jest lepszy, niż brak komunikatu, gdy błąd rzeczywiście wystąpił.

Czytelny język. Unikaj komunikatów typu „wystąpił błąd nr 15”.

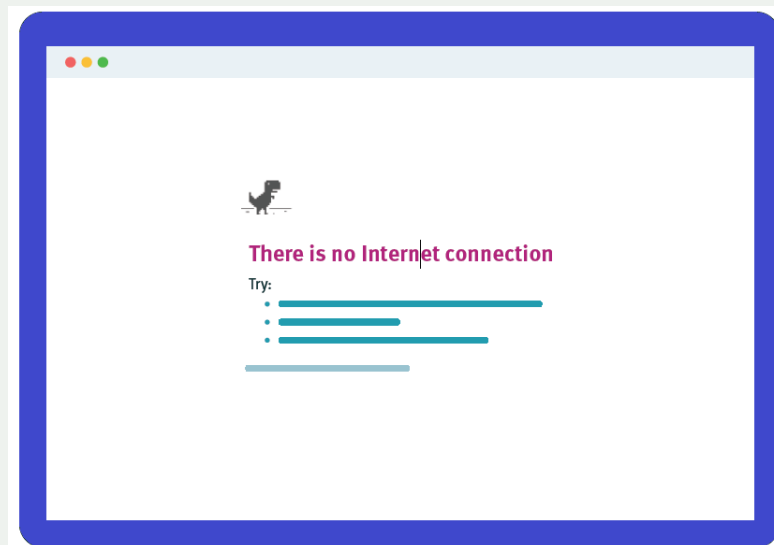
Uprzejmy ton. Nie sugeruj, że użytkownik jest głupi lub zrobił coś niewłaściwego, np. „nielegalna instrukcja”.

Dokładny opis błędu, zamiast ogólników w rodzaju „błąd syntaktyczny”.

Rada jak naprawić/poprawić sytuację.

UWAGA: W diagnostyce i korekcji błędu rzadko stosuje się elementy graficzne.

Wyjątkiem są systemy przedstawiane w postaci diagramów blokowych, na których można wyświetlać lokalizację błędu.



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

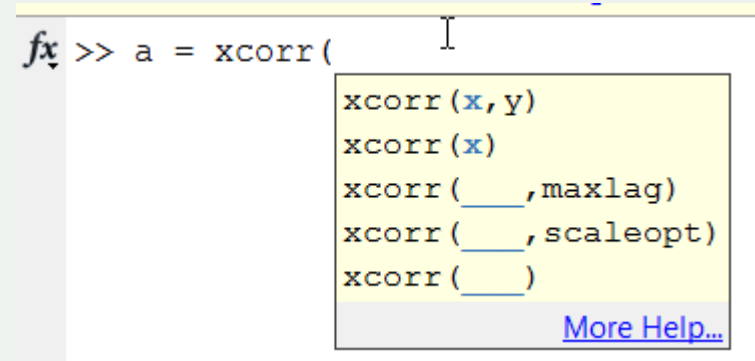
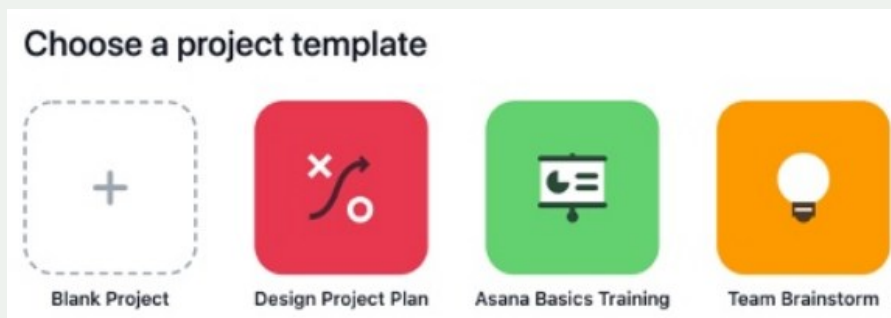
10. Dokumentacja (łatwo dostępna pomoc na żądanie)

Pomoc *proaktywna* i *reaktywna*.

Celem pomocy proaktywnej jest:

1. Szkolenie nowych użytkowników, np. tutoriale, wzorce, *wizards* (asystenci, kreatorzy), itp.
2. Pomoc początkującym użytkownikom w osiągnięciu biegłości.
3. Pomoc w nowych/zmienionych możliwościach lub opcjach systemu.

Przykłady pomocy proaktywnej:



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

10. Dokumentacja (łatwo dostępna pomoc na żądanie)

Pomoc *reaktywna* powinna być dostępna jedynie na wyraźne żądanie użytkownika (gdy napotka on problem). Powinna wyjaśnić problem, odpowiedzieć na pytania i ogólnie pomóc użytkownikowi w osiągnięciu poziomu eksperta.

Najbardziej typowe formy pomocy reaktywnej to:

- FAQs.
- Dokumentacja techniczna. Pamiętaj, że użytkownicy nie lubią czytać dokumentacji, a jeśli już muszą, to oczekują tam specjalistycznej wiedzy (a nie ogólnej).
- Moduły/sesje treningowe.

Choć pomoc reaktywna jest głównie w formie tekstowej, można ją wspomagać grafiką i filmami.

UWAGA: Idealny system to taki, który nie wymaga dodatkowych wyjaśnień, a więc rozmiar dokumentacji jest nieformalną miarą niedoskonałości systemu (z punktu widzenia użytkownika).

Zasady projektowania interfejsów graficznych

Iluzje optyczne (potencjalne utrudnienia albo ułatwienia ☺ w odbiorze informacji wizualnej)

Najważniejsze rodzaje iluzji:

- Postrzeganie rozmiarów.
- Postrzeganie jasności (kolorów).
- Postrzeganie głębi.

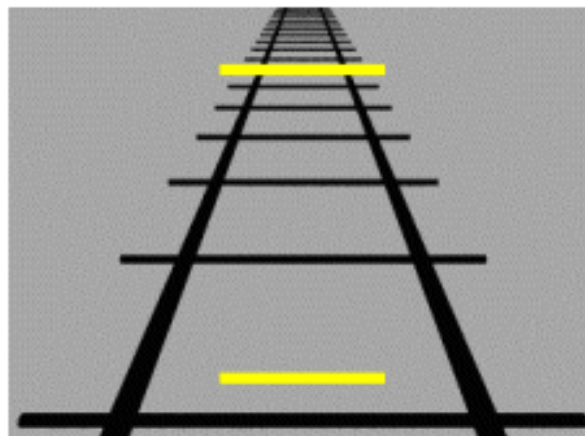
Inne rodzaje iluzji

- Postrzeganie (rozpoznawanie) kształtów.
- Postrzeganie struktur 3-wymiarowych (obiekty niemożliwe).
- Postrzeganie ruchu.

- Postrzeganie rozmiarów.



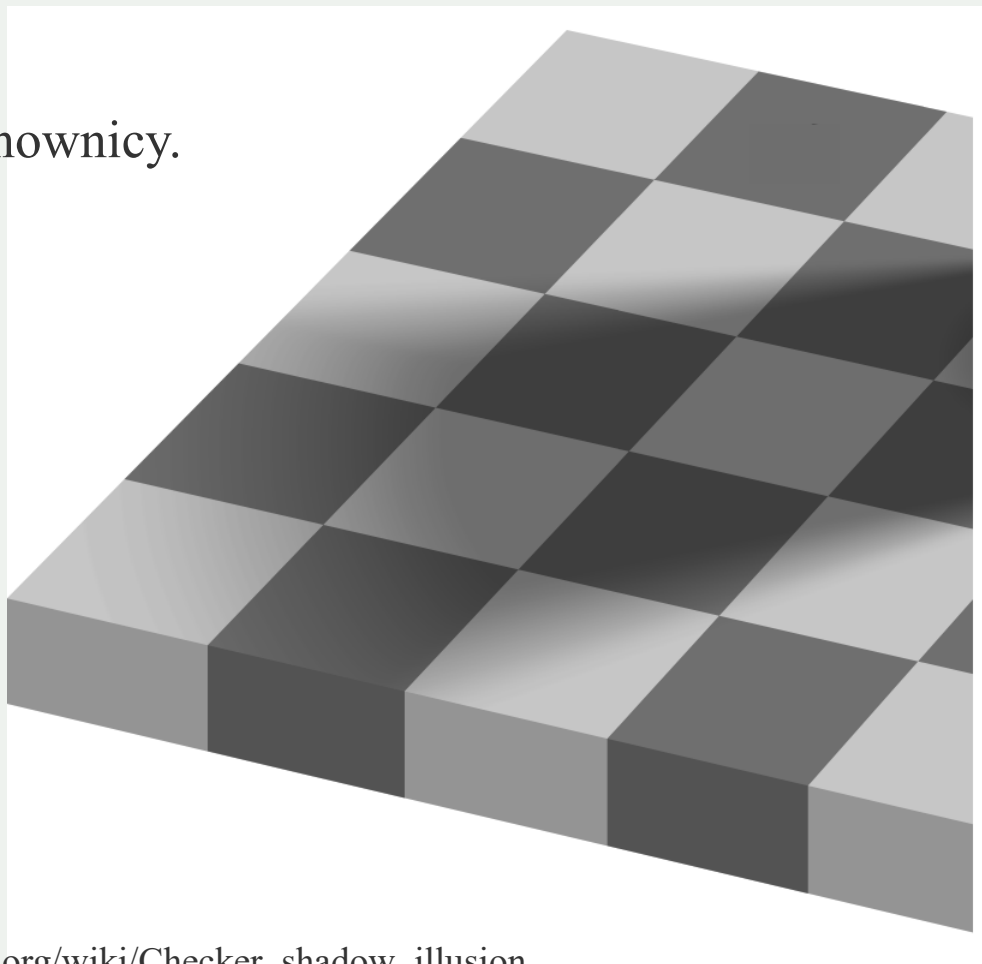
- Postrzeganie rozmiarów.



Iluzja Ponzo.

- Postrzeganie jasności (dotyczy również kolorów).

Iluzja szachownicy.

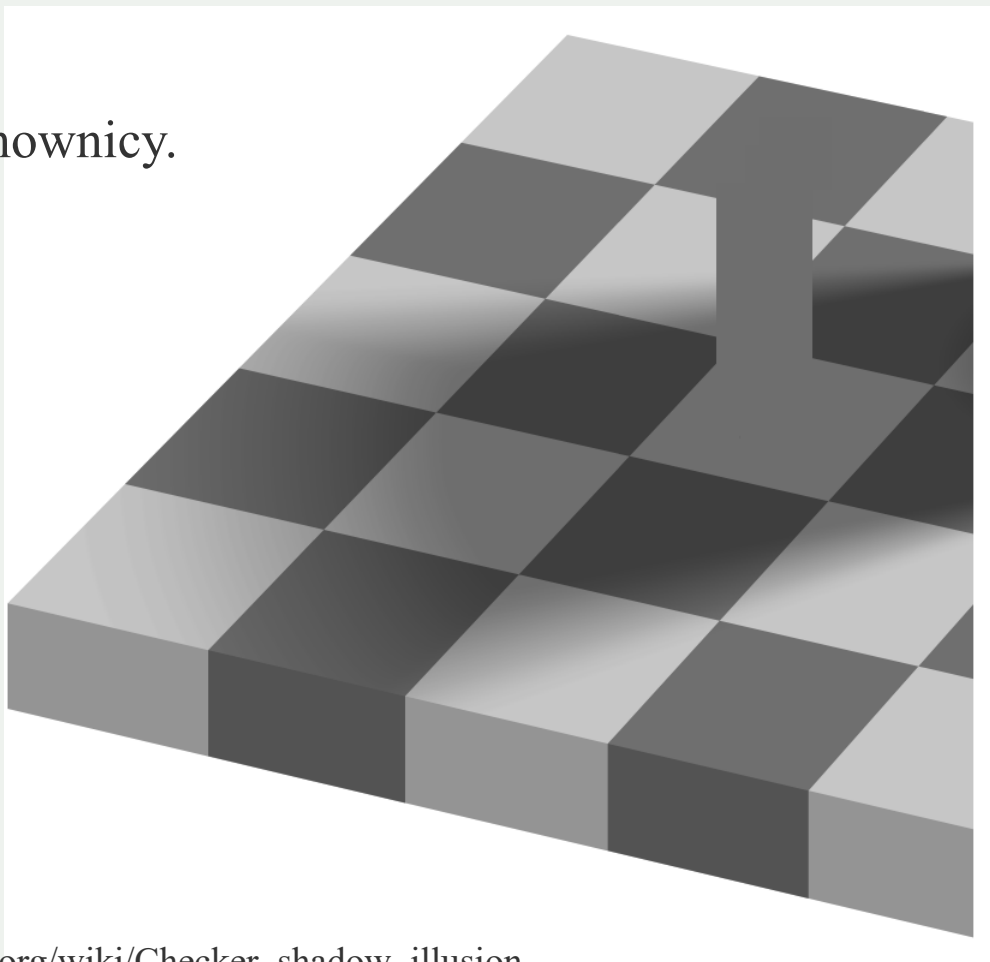


Edward H. Adelson

https://en.wikipedia.org/wiki/Checker_shadow_illusion

- Postrzeganie jasności (dotyczy również kolorów).

Iluzja szachownicy.



Edward H. Adelson

https://en.wikipedia.org/wiki/Checker_shadow_illusion

- Postrzeganie głębi.



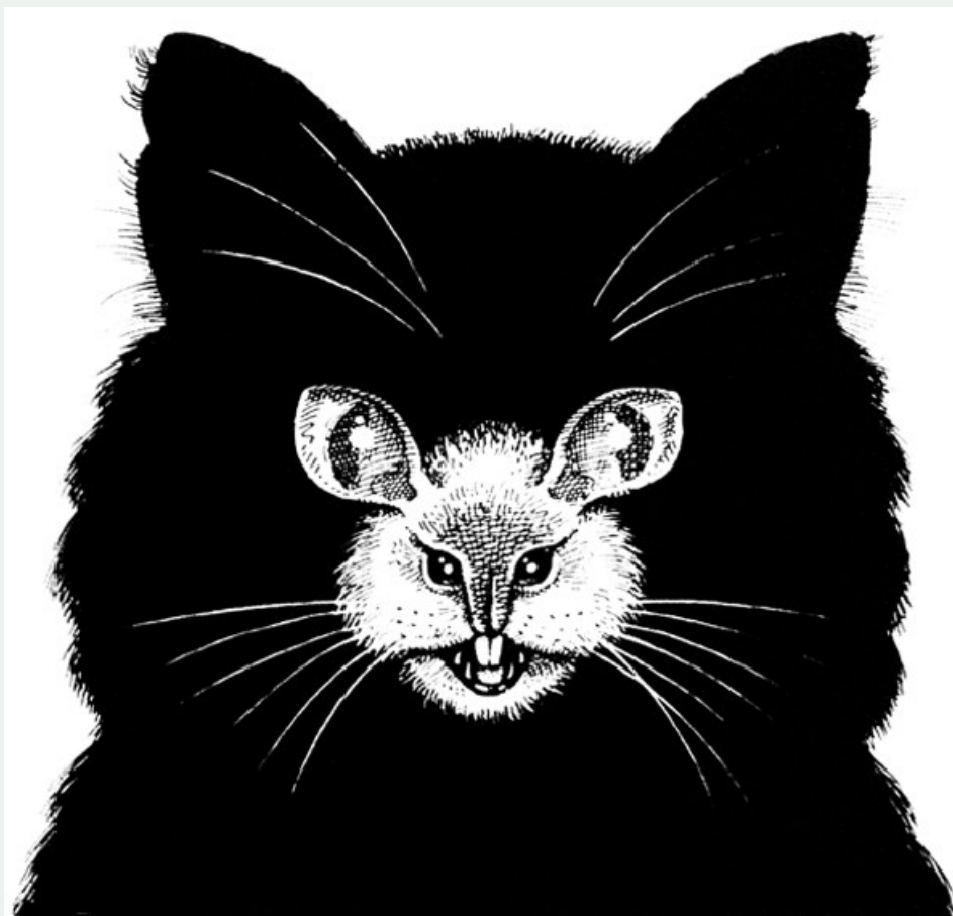
Iluzja „księżycowa”.

- Postrzeganie głębi.



Iluzja „księżycowa”.

- Postrzeganie (rozpoznawanie) kształtów.



Źródło: <https://designpress.com/inspiration/scary-optical-illusions/>

Uzupełnienie

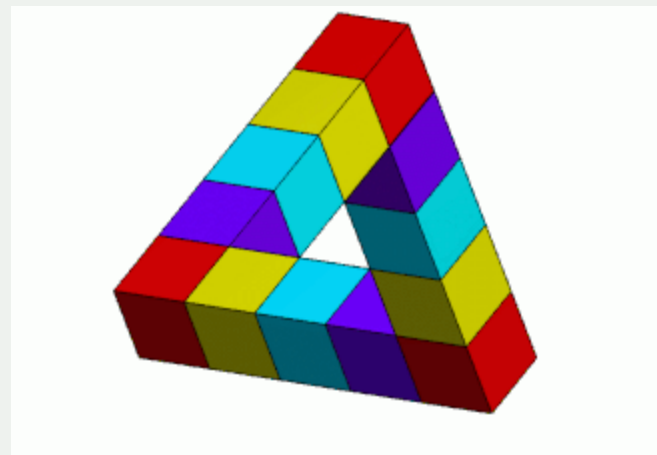
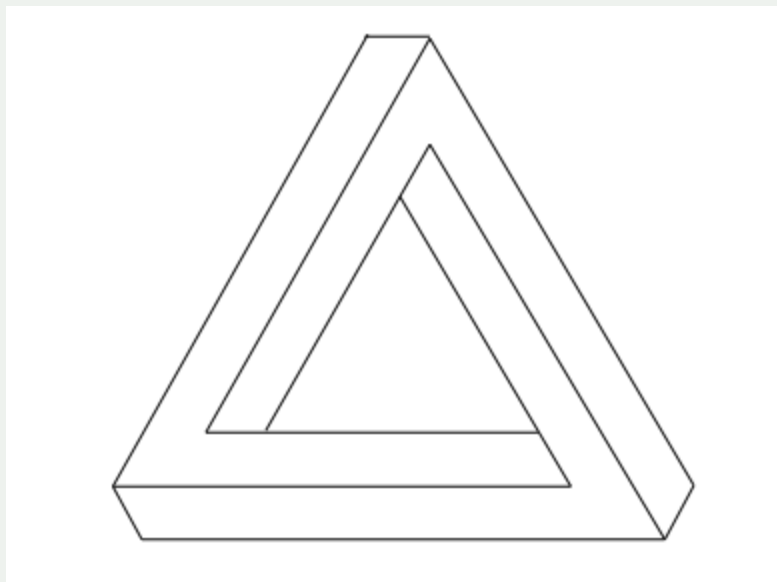
Ciekawe efekty graficzne można czasami uzyskać wklejając w grafikę 2D tzw. *obiekty niemożliwe* (*impossible objects*). Są to rzuty 2D struktur 3-wymiarowych, których fizyczne istnienie wydaje się niemożliwe.



Pierwsze próby pochodzą z wieków średnich (Pieter Breughel „Pejzaż z szubienicą”).

Uzupełnienie poprzedniego wykładu

W wielu przypadkach są to „możliwe” obiekty. Na rysunku widzimy tzw. trójkąt Penrose’a.



Źródło: https://en.wikipedia.org/wiki/Penrose_triangle

W świecie 3D może to być albo otwarta struktura (jak na animacji, choć jest w niej pewien błąd) albo trójkąt zbudowany z belek o krzywoliniowych krawędziach, które w danym rzucie ułożone są w taki sposób, że te krawędzie widać jako odcinki proste.

Zasady projektowania interfejsów (graficznych)

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika